```
=== FILE: ./sandbox/voice_jarvis.py ===
import threading
import time
from collections import OrderedDict

class TTLCache:
    def __init__(self, capacity: int, default_ttl: float, cleanup_interval:\033[17D\033[K
cleanup_interval: float):
        self.capacity = capacity
        self.default_ttl = default_ttl
        self.cleanup_interval = cleanup_interval
        self.cache = OrderedDict()
        self.lock = threading.Lock()
        self.hits = 0
        self.misses = 0
        self.evictions = 0
        self.expirations = 0
        self.on_evict = None

    def get(self, key):
        with self.lock:
            if key in self.cache:
                self.hits += 1
                self._update_ttl(key)
                return self.cache[key]
            else:
                self.misses += 1
                return None

    def put(self, key, value, ttl=None):
        with self.lock:
            if key in self.cache:
                self.evictions += 1
                self.cache.pop(key)
            self._update_ttl(key, ttl or self.default_ttl)
            self.cache[key] = value
            if len(self.cache) > self.capacity:
                self._evict_lru()

    def _update_ttl(self, key, ttl=None):
        if ttl is not None:
            self.cache.move_to_end(key, last=False)
        else:
            self.cache.move_to_end(key)

    def _evict_lru(self):
        with self.lock:
            while len(self.cache) > self.capacity:
                key, _ = self.cache.popitem(last=False)
                self.expirations += 1
                if self.on_evict:
                    self.on_evict(key, self.cache[key], "lru")

    def start_cleanup(self):
        while True:
            with self.lock:
                if time.time() - self.last_cleanup > self.cleanup_interval:\033[22D\033[K
self.cleanup_interval:
                    self._evict_expired()
            time.sleep(self.cleanup_interval)

    def _evict_expired(self):
        with self.lock:
            for key in list(self.cache.keys()):
                if time.time() - self.cache[key] > self.default_ttl:
                    self.evictions += 1
                    if self.on_evict:
                        self.on_evict(key, self.cache[key], "ttl")
                    self.cache.pop(key)
```

```
    def get_stats(self):
        with self.lock:
            return {
                "hits": self.hits,
                "misses": self.misses,
                "evictions": self.evictions,
                "expirations": self.expirations,
                "size": len(self.cache),
                "hit_ratio": self.hits / (self.hits + self.misses) if self.\033[5D\033[K
self.hits + self.misses > 0 else 0
            }

    def set_on_evict(self, callback):
        self.on_evict = callback

# Ð¢ÐµÑ\201Ñ\202
if __name__ == "__main__":
    cache = TTLCache(capacity=100, default_ttl=0.5, cleanup_interval=0.1)
    cache.set_on_evict(lambda key, value, reason: print(f"Evicted: {key} du\033[2D\033[K
due to {reason}"))

    threads = []
    for _ in range(8):
        thread = threading.Thread(target=lambda: [
            cache.get(i % 50),
            cache.put(i % 50, i, 0.05 + 0.15 * (i % 50))
            for i in range(500)
        ])
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    stats = cache.get_stats()
    assert stats["hits"] + stats["misses"] == 4000
    assert stats["size"] <= 100
    assert not cache.on_evict_called  # Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð° Ñ\202Ð¾Ð³Ð¾, Ñ\207Ñ\202
Ð¾ on_evict Ð½Ðµ Ð²Ñ\213Ð•Ñ\213\033[4D\033[K
Ð²Ñ\213Ð•Ñ\213Ð²Ð°Ð»Ñ\201Ñ\217 Ð¿Ð¾Ð´ Ð»Ð¾Ð¾Ð¼Ð¼

    print(stats)-e


=== FILE: ./sandbox/report.txt ===
2026-10-25 13:22:44-e


=== FILE: ./sandbox/backup_test/file_1.txt ===
100-e


=== FILE: ./sandbox/backup_test/file_2.txt ===
62-e


=== FILE: ./sandbox/backup_test/file_3.txt ===
55-e


=== FILE: ./sandbox/backup_test/file_4.txt ===
33-e


=== FILE: ./sandbox/backup_test/file_5.txt ===
68-e


=== FILE: ./sandbox/backup_test/total_sum.txt ===
318-e
```

```
=== FILE: ./sandbox.py ===
import os
import subprocess
import shutil
import time
import re

# --- Ð\235Ð\220Ð¡Ð¢Ð Ð\236Ð\231Ð\232Ð\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
SANDBOX_DIR = os.path.join(FACTORY_DIR, "sandbox")
SOURCE_FILE = os.path.join(FACTORY_DIR, "draft_final.py")
TARGET_FILE = os.path.join(SANDBOX_DIR, "voice_jarvis.py")
TIMEOUT = 10

# --- Ð§Ð\225Ð Ð\235Ð«Ð\231 Ð¡Ð\237Ð\230Ð¡Ð\236Ð\232 Ð\236Ð\237Ð\220Ð¡Ð\235Ð«Ð¥ Ð\232Ð
\236Ð\234Ð\220Ð\235Ð\224 ---
BLACKLIST = [
    r"os\.remove", r"os\.unlink", r"os\.rmdir", r"os\.removedirs",
    r"shutil\.rmtree", r"shutil\.move", r"shutil\.copy",
    r"subprocess\.run.*rm\s", r"subprocess\.run.*del\s",
    r"os\.system", r"eval\(", r"exec\(",
    r"__import__\('os'\)", r"__import__\('sys'\)",
    r"open\(.*['\"]\/", r"open\(.*['\"]~",
    r"requests\.", r"urllib\.", r"socket\."
]

def is_safe(code):
    """Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ñ\217ÐµÑ\202 Ð°Ð¾Ð´ Ð½Ð° Ð»Ð¸Ñ\207Ð¸Ðµ Ð¾Ð¿Ð°Ñ\201Ð½Ñ\213
Ñ\205 Ð°Ð¾Ð¼Ð°Ð½Ð´."""
    for pattern in BLACKLIST:
        if re.search(pattern, code, re.IGNORECASE):
            return False, f"Ð\236Ð±Ð½Ð°Ñ\200Ñ\203Ð¶ÐµÐ½Ð° Ð¾Ð¿Ð°Ñ\201Ð½Ð°Ñ\217 Ð°Ð¾Ð¼Ð°Ð½
Ð´Ð°: {pattern}"
    return True, "OK"

def main():
    print("[SANDBOX] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ñ\203Ð¼Ð½Ð¾Ð¹ Ð¿ÐµÑ\201Ð¾Ñ\207Ð½Ð¸Ñ\206Ñ\213..
.")

    if not os.path.exists(SANDBOX_DIR):
        os.makedirs(SANDBOX_DIR, exist_ok=True)

    if not os.path.exists(SOURCE_FILE):
        print(f"[SANDBOX] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¸Ñ\201Ñ\205Ð¾Ð´Ð½Ð¾Ð³Ð¾ Ñ\204Ð°Ð¹Ð» Ð½Ðµ
Ð½Ð°Ð¹Ð´ÐµÐ½: {SOURCE_FILE}")
        return

    # 1. Ð§Ð¸Ñ\202Ð°ÐµÐ¼ Ð°Ð¾Ð´ Ð¸ Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ñ\217ÐµÐ¼ Ð½Ð° Ð±ÐµÐ•Ð¾Ð¿Ð°Ñ\201Ð½Ð¾
Ñ\201Ñ\202Ñ\214
    try:
        with open(SOURCE_FILE, "r", encoding="utf-8") as f:
            code = f.read()
    except Exception as e:
        print(f"[SANDBOX] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ñ\207Ñ\202ÐµÐ½Ð¸Ñ\217 Ñ\204Ð°Ð¹Ð»Ð°: {e}")
        return

    safe, reason = is_safe(code)
    if not safe:
        print(f"[SANDBOX] Ð\221Ð\233Ð\236Ð\232Ð\230Ð Ð\236Ð\222Ð\232Ð\220! {reason}")
        # Ð¡Ð¾Ñ\205Ñ\200Ð°Ð½Ñ\217ÐµÐ¼ Ð»Ð¾Ð³ Ð¾Ñ\210Ð¸Ð±Ð¾Ðº Ð±ÐµÐ•Ð¾Ð¿Ð°Ñ\201Ð½Ð¾Ñ\201Ñ
\202Ð¸
        with open(os.path.join(SANDBOX_DIR, "sandbox_error.log"), "w", encoding="utf-8")
as f:
            f.write(f"Ð\236Ð¨Ð\230Ð\221Ð\232Ð\220 Ð\221Ð\225Ð\227Ð\236Ð\237Ð\220Ð¡Ð\235Ð
\236Ð¡Ð¢Ð\230: {reason}\nÐ\232Ð¾Ð´ Ð•Ð°Ð±Ð»Ð¾Ð°Ð¸Ñ\200Ð¾Ð²Ð°Ð½.")
        return

    # 2. Ð\232Ð¾Ð¿Ð¸Ñ\200Ñ\203ÐµÐ¼ Ð² Ð¿ÐµÑ\201Ð¾Ñ\207Ð½Ð¸Ñ\206Ñ\203
```

```
    try:
        shutil.copy2(SOURCE_FILE, TARGET_FILE)
        print("[SANDBOX] Đ\232Đ¾Đ´ Đ±ĐµĐ•Đ¾Đ¿Đ°Ñ\201ĐµĐ½. Đ¤Đ°Đ¹Đ» Ñ\201Đ°Đ¾Đ¿Đ¸Ñ\200Đ¾Đ²
Đ°Đ½ Đ² Đ¿ĐµÑ\201Đ¾Ñ\207Đ½Đ¸Ñ\206Ñ\203.")
    except Exception as e:
        print(f"[SANDBOX] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ°Đ¾Đ¿Đ¸Ñ\200Đ¾Đ²Đ°Đ½Đ¸Ñ\217: {e}")
        return

    # 3. Đ\227Đ°Đ¿Ñ\203Ñ\201Đ°Đ°ĐµĐ¼ Đ°Đ¾Đ´
    print("[SANDBOX] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Ñ\201Đ°Ñ\200Đ¸Đ¿Ñ\202Đ°...")
    start_time = time.time()
    status = "unknown"
    error_text = ""

    try:
        process = subprocess.Popen(
            ["python", TARGET_FILE],
            cwd=SANDBOX_DIR,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True
        )

        try:
            stdout, stderr = process.communicate(timeout=TIMEOUT)
        except subprocess.TimeoutExpired:
            process.kill()
            process.wait()
            status = "Đ\236Đ¨Đ\230Đ\221Đ\232Đ\220 (Đ•Đ°Đ²Đ¸Ñ\201, Ñ\203Đ±Đ¸Ñ\202 SIGKILL)
"
            error_text = f"Đ\237Ñ\200Đ¾Ñ\206ĐµÑ\201Ñ\201 Đ½Đµ Đ•Đ°Đ²ĐµÑ\200Ñ\210Đ¸Đ»Ñ\201
Ñ\217 Đ•Đ° {TIMEOUT} Ñ\201ĐµĐ°Ñ\203Đ½Đ´."

        elapsed = round(time.time() - start_time, 2)

        if status == "unknown":
            if process.returncode == 0:
                status = "Đ£Đ¡Đ\237Đ¥ (Đ•Đ°Đ²ĐµÑ\200Ñ\210Đ¸Đ»Ñ\201Ñ\217 Ñ\201 Đ°Đ¾Đ´
Đ¾Đ¼ 0)"
            else:
                status = f"Đ\236Đ¨Đ\230Đ\221Đ\232Đ\220 (Ñ\203Đ¿Đ°Đ» Ñ\201 Đ°Đ¾Đ´Đ¾Đ¼ {pro
cess.returncode})"
                error_text = stderr

    except Exception as e:
        print(f"[SANDBOX] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Đ•Đ°Đ¿Ñ\203Ñ\201Đ°Đµ: {e}")
        return

    # 4. Đ\237Đ¸Ñ\210ĐµĐ¼ Đ»Đ¾Đ³Đ¸
    log_suffix = "success" if "Đ£Đ¡Đ\237Đ¥" in status else "error"
    log_path = os.path.join(SANDBOX_DIR, f"sandbox_{log_suffix}.log")

    try:
        with open(log_path, "w", encoding="utf-8") as f:
            f.write(f"Đ¡Đ°Ñ\200Đ¸Đ¿Ñ\202 Ñ\200Đ°Đ±Đ¾Ñ\202Đ°Đ» {elapsed} Ñ\201ĐµĐ°.\nĐ¡Ñ
\202Đ°Ñ\202Ñ\203Ñ\201: {status}\n")
            if error_text:
                f.write(f"\nĐ\236Ñ\210Đ¸Đ±Đ°Đ°:\n{error_text}\n")
    except Exception as e:
        print(f"[SANDBOX] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ•Đ°Đ¿Đ¸Ñ\201Đ¸ Đ»Đ¾Đ³Đ°: {e}")

    print(f"[SANDBOX] Đ\236Ñ\202Ñ\207ĐµÑ\202: Ñ\200Đ°Đ±Đ¾Ñ\202Đ°Đ» {elapsed} Ñ\201ĐµĐ°.,
Ñ\201Ñ\202Đ°Ñ\202Ñ\203Ñ\201: {status}")
    if error_text:
        print(f"[SANDBOX] Đ\236Ñ\210Đ¸Đ±Đ°Đ°:\n{error_text}")

if __name__ == "__main__":
    main()
-e
```

```
=== FILE: ./coder.py ===
import os
import subprocess
import re

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
FEEDBACK_FILE = os.path.join(FACTORY_DIR, "feedback.txt")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft_1.py")
MODEL = "qwen2.5-coder:3b"

def clean_code(text):
    match = re.search(r'```(?:python)?\s*(.*?)```', text, re.DOTALL)
    if match: return match.group(1).strip()
    return text.strip()

def main():
    if not os.path.exists(TASK_FILE): return
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()

    feedback = ""
    if os.path.exists(FEEDBACK_FILE):
        with open(FEEDBACK_FILE, "r", encoding="utf-8") as f:
            feedback = f.read().strip()

    prompt = f"Р\227Р°Р´Р°Ñ\207Р°: {task}\n"
    if feedback:
        prompt += f"\nР\222Р\235Р\230Р\234Р\220Р\235Р\230Р\225! Р\237Ñ\200РµÐ´Ñ\213Ð´Ñ
\203Ñ\211Р°Ñ\217 Р¿Ð¾Ð¿Ñ\213Ñ\202Р°Р° Ñ\203Р¿Р°Р»Р°. Р\230Ñ\201Р¿Ñ\200Р°Ð²Ñ\214 Ñ\215Ñ
\202Ð¸ Ð¾Ñ\210Ð¸Ð±Р°Ð¸:\n{feedback}\n"

    prompt += "Р\235Р°Р¿Ð¸Ñ\210Ð¸ Р°Ð¾Ð´ Ð½Р° Python, Ð¸Ñ\201Р¿Ð¾Ð»Ñ\214Р•Ñ\203Ð¹ Ñ\202Ð¾
Ð»Ñ\214Р°Ð¾ Ñ\201Ñ\202Р°Ð½Ð´Р°Ñ\200Ñ\202Ð½Ñ\213Ðµ Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð¸. Р\235Р°Р¿Ð¸Ñ
\210Ð¸ Ñ\202Ð¾Ð»Ñ\214Р°Ð¾ Р°Ð¾Ð´."

    print(f"[CODER] Р\236Ñ\202Р¿Ñ\200Р°Ð²Ð°Ð° Р•Р°Р¿Ñ\200Ð¾Ñ\201Р° Ð² Ollama ({MODEL})...
")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt, "--num-predict", "500"],
            capture_output=True, text=True, timeout=600
        )
        clean = clean_code(result.stdout)
        with open(DRAFT_FILE, "w", encoding="utf-8") as f:
            f.write(clean)
        print("[CODER] Р\232Ð¾Ð´ Ñ\201Ð¾Ñ\205Ñ\200Р°Ð½ÐµÐ½ Ð² draft_1.py")
    except Exception as e:
        print(f"[CODER] Р\236Ñ\210Ð¸Ð±Р°Р°: {e}")

if __name__ == "__main__":
    main()
-e


=== FILE: ./task.txt ===
Р\235Р°Р¿Ð¸Ñ\210Ð¸ Ð½Р° Python (Ñ\202Ð¾Ð»Ñ\214Р°Ð¾ Ñ\201Ñ\202Р°Ð½Ð´Р°Ñ\200Ñ\202Ð½Р°Ñ\217
Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð°) Р¿Ð¾Ñ\202Ð¾Ð¾Ð±ÐµÐ•Ð¾Ð¿Ð°Ñ\201Ð½Ñ\213Ð¹ TTL-Р°Ñ\215Ñ\210 Ñ\201
 LRU-Ð²Ñ\213Ñ\202ÐµÑ\201Ð½ÐµÐ½Ð¸ÐµÐ¼, Ñ\204Ð¾Ð½Ð¾Ð²Ð¾Ð¹ Ð¾Ñ\207Ð¸Ñ\201Ñ\202Р°Ð¾Ð¹, callba
ck'Ð¾Ð¼ Ð½Р° Ð²Ñ\213Ñ\202ÐµÑ\201Ð½ÐµÐ½Ð¸Ðµ Ð¸ Ð¼ÐµÑ\202Ñ\200Ð¸Р°Р°Ð¼Ð¸. Р¢Ñ\200ÐµÐ±Ð¾Ð²Ð°
Ð½Ð¸Ñ\217: 1) Р\232Ð»Р°Ñ\201Ñ\201 TTLCache(capacity: int, default_ttl: float, cleanup_int
erval: float). 2) Р\234ÐµÑ\202Ð¾Ð´Ñ\213 get(key) Ð¸ put(key, value, ttl=None) Ð¿Ð¾Ñ\202Ð¾
Ð°Ð±ÐµÐ•Ð¾Ð¿Ð°Ñ\201Ð½Ñ\213, Ð¸Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ñ\203Ñ\216Ñ\202 threading.Lock (Ð¸Ð»Ð¸
RLock â\200\224 Ð¾Ð±Ð¾Ñ\201Ð½Ñ\203Ð¹ Ð²Ñ\213Ð±Ð¾Ñ\200 Ð² Р°Ð¾Ð¼Ð¾ÐµÐ½Ð½Ñ\202Р°Ñ\200Ð¸Ð¸). 3
) LRU-Ð²Ñ\213Ñ\202ÐµÑ\201Ð½ÐµÐ½Ð¸Ðµ: Ð¿Ñ\200Ð¸ Ð¿Ñ\200ÐµÐ²Ñ\213Ñ\210ÐµÐ½Ð¸Ð¸ capacity Ñ
\203Ð´Р°Ð»Ñ\217ÐµÑ\202Ñ\201Ñ\217 Ñ\201Р°Ð¼Ñ\213Ð¹ Ð´Р°Ð²Ð½Ð¾ Ð¸Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ð¾Ð²Ð°Ð½Ð½Ñ\213Ð¹ Ñ\215Ð»ÐµÐ¼ÐµÐ½Ñ\202; Ñ\200Р°Ð•Ñ\200ÐµÑ\210Ñ\221Ð½ OrderedDict, Ð•Р°Ð¿Ñ\200ÐµÑ
\211ÐµÐ½Ñ\213 functools.lru_cache Ð¸ Ð³Ð¾Ñ\202Ð¾Ð²Ñ\213Ðµ LRU-Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð¸. 4)
 TTL Ð½Р° Ð•Р°Ð¿Ð¸Ñ\201Ñ\214: put Ñ\201 ttl Ð¿ÐµÑ\200ÐµÐ¾Ð¿Ñ\200ÐµÐ´ÐµÐ»Ñ\217ÐµÑ\202 defa
```

```
ult_ttl; Ð¿Ñ\200Ð¾Ñ\201Ñ\200Ð¾Ñ\207ÐµÐ½Ð½Ð°Ñ\217 Ð•Ð°Ð¿Ð¸Ñ\201Ñ\214 Ð¿Ñ\200Ð¸ get Ñ\201Ñ
\207Ð¸Ñ\202Ð°ÐµÑ\202Ñ\201Ñ\217 miss Ð¸ Ñ\203Ð´Ð°Ð»Ñ\217ÐµÑ\202Ñ\201Ñ\217; Ð´Ð¾Ð¿Ð¾Ð»Ð½Ð¸Ñ
\202ÐµÐ»Ñ\214Ð½Ð¾ Ñ\200Ð°Ð±Ð¾Ñ\202Ð°ÐµÑ\202 Ñ\204Ð¾Ð½Ð¾Ð²Ñ\213Ð¹ daemon-Ð¿Ð¾Ñ\202Ð¾Ð°, Ð°
Ð¾Ñ\202Ð¾Ñ\200Ñ\213Ð¹ Ñ\200Ð°Ð• Ð² cleanup_interval Ñ\201ÐµÐ°Ñ\203Ð½Ð´ Ñ\203Ð´Ð°Ð»Ñ\217Ðµ
Ñ\202 Ð¸Ñ\201Ñ\202Ñ\221Ð°Ñ\210Ð¸Ðµ Ð•Ð°Ð¿Ð¸Ñ\201Ð¸, Ð½Ð¾ Ð½ÐµÐ´Ð¾Ñ\201Ð¿Ñ\200Ð¶Ð¸Ñ\202 Ð»Ð¾Ð° Ð
²Ð¾ Ð²Ñ\200ÐµÐ¼Ñ\217 Ð¸Ñ\202ÐµÑ\200Ð°Ñ\206Ð¸Ð¸ Ð¿Ð¾ Ð²Ñ\201ÐµÐ¼ Ð°Ð»Ñ\216Ñ\207Ð°Ð¼ â\200
\224 Ñ\201Ð½Ð¸Ð¼Ð°Ð¹ Ð°Ð»Ñ\216Ñ\207Ð¸ Ð¿Ð°Ñ\207Ð°Ð¹Ð¹ Ð¿Ð¾Ð´ Ð»Ð¾Ð°Ð¼Ð¼, Ñ\203Ð´Ð°Ð»Ñ\217
Ð¹ Ð¸ Ð²Ñ\213Ð•Ñ\213Ð²Ð°Ð¹ callback Ð²Ð½ÐµÑ Ð»Ð¾Ð°Ð°, Ð¾Ð±Ð¾Ñ\201Ð½Ñ\203Ð¹ Ñ\215Ñ\202Ð¾ Ð²
Ð°Ð¾Ð¼Ð¼ÐµÑ\202Ð°Ñ\200Ð¸Ð¸. 5) Callback on_evict(key, value, reason), Ð³Ð´Ðµ reason â
\200\224 "lru" Ð¸Ð»Ð¸ "ttl"; callback Ð½ÐµÑ Ð´Ð¾Ð»Ð¶ÐµÐ½ Ð²Ñ\213Ð•Ñ\213Ð²Ð°Ñ\202Ñ\214Ñ\201
Ñ\217 Ð¿Ð¾Ð´ Ð»Ð¾Ð°Ð¼Ð¼ (Ð¸Ð½Ð°Ñ\207Ðµ Ð´ÐµÐ´Ð»Ð¾Ð°, ÐµÑ\201Ð»Ð¸ callback Ñ\201Ð°Ð¼ Ñ\202
Ñ\200Ð¾Ð³Ð°ÐµÑ\202 Ð°Ñ\215Ñ\210) â\200\224 Ñ\201Ð¾Ð±ÐµÑ\200Ð¸ Ð¶ÐµÑ\200Ñ\202Ð²Ñ\213 Ð¿Ð¾Ð
´ Ð»Ð¾Ð°Ð¾Ð¼, Ð²Ñ\213Ð•Ð¾Ð²Ð¸ Ð¿Ð¾Ñ\201Ð»Ðµ Ð¾Ñ\202Ð¿Ñ\203Ñ\201Ð°Ð²Ð½Ð¸Ñ\217; eviction Ð´
Ð¾Ð»Ð¶Ð½Ñ Ñ\203Ñ\207Ð¸Ñ\202Ñ\213Ð²Ð°Ñ\202Ñ\214Ñ\201Ñ\217 Ñ\200Ð¾Ð²Ð½Ð¾ Ð¾Ð´Ð¸Ð½ Ñ\200Ð°Ð
• (Ð½Ðµ Ð¿Ð¾Ñ\202ÐµÑ\200Ñ\217Ñ\202Ñ\214 Ð¸ Ð½Ðµ Ð•Ð°Ð´Ð²Ð¾Ð¸Ñ\202Ñ\214), Ð° put Ñ\201 Ð¿Ð
µÑ\200ÐµÐ•Ð°Ð¿Ð¸Ñ\201Ñ\214Ñ\216 Ñ\201Ñ\203Ñ\211ÐµÑ\201Ñ\202Ð²Ñ\203Ñ\216Ñ\211ÐµÐ³Ð¾ Ð°Ð»Ñ
\216Ñ\207Ð° Ð½Ðµ Ð´Ð¾Ð»Ð¶ÐµÐ½ Ñ\201Ñ\207Ð¸Ñ\202Ð°Ñ\202Ñ\214Ñ\201Ñ\217 eviction. 6) Ð\234Ð
µÑ\202Ñ\200Ð¸Ð°Ð¸ hits, misses, evictions, expirations; get_stats() Ð²Ð¾Ð•Ð²Ñ\200Ð°Ñ\211Ð
°ÐµÑ\202 Ð¸Ñ\205 + size + hit_ratio. 7) Ð¢ÐµÑ\201Ñ\202: 8 Ð¿Ð¾Ñ\202Ð¾Ð°Ð² Ã\227 500 Ð¾Ð
¿ÐµÑ\200Ð°Ñ\206Ð¸Ð¹ get/put Ñ\201 Ð°Ð»Ñ\216Ñ\207Ð°Ñ¼ Ð¸Ð• Ð¿Ñ\203Ð»Ð° Ð½Ð° 50 Ð°Ð»Ñ\216
Ñ\207ÐµÐ¹ (Ñ\207Ñ\202Ð¾Ð±Ñ\213 Ð±Ñ\213Ð»Ð¸ Ð¸ hits, Ð¸ evictions), Ñ\207Ð°Ñ\201Ñ\202Ñ\214
 put â\200\224 Ñ\201 Ð°Ð¾Ñ\200Ð¾Ñ\202Ð°Ð¸Ñ¼ TTL (0.05â\200\2230.2 Ñ\201), Ð°Ñ\215Ñ\210 ca
pacity=100, default_ttl=0.5, cleanup_interval=0.1. Ð\237Ð¾Ñ\201Ð»Ðµ join Ð¿Ñ\200Ð¾Ð²ÐµÐ
\200Ñ\214 Ð¸Ð½Ð²Ð°Ñ\200Ð¸Ð°Ð½Ñ\202Ñ\213: hits + misses Ñ\200Ð°Ð²Ð½Ð¾ Ñ\201Ñ\203Ð¼Ð¼Ð°Ñ
\200Ð½Ð¾Ð¼Ñ\203 Ñ\207Ð¸Ñ\201Ð»Ñ\203 get Ð¿Ð¾ Ð²Ñ\201ÐµÐ¼ Ð¿Ð¾Ñ\202Ð¾Ð°Ð°Ð¼; size <= capac
ity; on_evict Ð½Ð¸ Ñ\200Ð°Ð•Ñ\203 Ð½Ðµ Ð±Ñ\213Ð» Ð²Ñ\213Ð•Ð²Ð°Ð½ Ð¿Ð¾Ð´ Ð»Ð¾Ð°Ð¼Ð¼ (Ñ\204
Ð»Ð°Ð³ Ñ\207ÐµÑ\200ÐµÐ• threading.local). Ð\222Ñ\213Ð²ÐµÐ´Ð¸ Ñ\201Ñ\202Ð°Ñ\202Ð¸Ñ\201Ñ
\202Ð¸Ð°Ñ\203 Ð¸ assert Ð½Ð° Ð²Ñ\201Ðµ Ñ\202Ñ\200Ð¸ Ð¸Ð½Ð²Ð°Ð°Ñ\200Ð¸Ð°Ð½Ñ\202Ð°. Ð\222 Ð°Ð
¾Ð¼Ð¼ÐµÐ½Ñ\202Ð°Ñ\200Ð¸Ñ\217Ñ\205 Ð¾Ð±Ñ\212Ñ\217Ñ\201Ð½Ð¸: Ð¿Ð¾Ñ\207ÐµÐ¼Ñ\203 Ð²Ñ\213Ð±Ñ
\200Ð°Ð½ Lock/RLock, Ð°Ð°Ð° Ñ\204Ð¾Ð½Ð¾Ð²Ñ\213Ð¹ cleaner Ð¸Ð•Ð±ÐµÐ³Ð°ÐµÑ\202 Â«Ñ\201Ñ\202
Ð¾Ð¿Ð° Ð¼Ð¸Ñ\200Ð°Â», Ð°Ð°Ð° Ð¾Ð±ÐµÑ\201Ð¿ÐµÑ\207ÐµÐ½Ð° Ð°Ñ\202Ð¾Ð¼Ð°Ñ\200Ð½Ð¾Ñ\201Ñ\202Ñ
\214 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð¸ TTL + Ð¾Ð±Ð½Ð¾Ð²Ð»ÐµÐ½Ð¸Ñ\217 LRU + Ñ\201Ñ\207Ñ\221Ñ\202Ñ\207
Ð¸Ð°Ð¾Ð² Ð² get, Ð¸ Ð¿Ð¾Ñ\207ÐµÐ¼Ñ\203 callback Ð²Ñ\213Ð•Ñ\213Ð²Ð°ÐµÑ\202Ñ\201Ñ\217 Ð²Ð½Ð
µ Ð»Ð¾Ð°Ð°. Ð\236Ð±Ñ\212Ñ\221Ð¼ â\200\224 Ð¾Ð°Ð»Ð¾ 120â\200\223150 Ñ\201Ñ\202Ñ\200Ð¾Ð°
Ñ\201 Ñ\202ÐµÑ\201Ñ\202Ð¾Ð¼.
-e


=== FILE: ./draft.py ===
def count_words(words):
    word_count = {}
    for word in words:
        if word in word_count:
            word_count[word] += 1
        else:
            word_count[word] = 1
    return word_count-e


=== FILE: ./tester.py ===
import os
import subprocess
import re

# --- Ð\235Ð\220Ð¡Ð¢Ð Ð\236Ð\231Ð\232Ð\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft.py")
REPORT_FILE = os.path.join(FACTORY_DIR, "report.txt")
COMMANDS_FILE = os.path.join(FACTORY_DIR, "termux_commands.txt")
DOCS_FILE = os.path.join(FACTORY_DIR, "docs", "termux_api.md")
MODEL = "qwen2.5-coder:3b"

# Ð\221Ð°Ð•Ð¾Ð²Ñ\213Ð¹ Ñ\201Ð¿Ñ\200Ð°Ð²Ð¾Ñ\207Ð½Ð¸Ð° (ÐµÑ\201Ð»Ð¸ Ñ\204Ð°Ð¹Ð»Ð° Ð½ÐµÑ\202
)
DEFAULT_COMMANDS = """- termux-torch on  |  ["termux-torch", "on"]
- termux-torch off  |  ["termux-torch", "off"]
- termux-brightness <0-255>  |  ["termux-brightness", "<0-255>"]
- termux-volume music <0-15>  |  ["termux-volume", "music", "<0-15>"]
- termux-camera-photo -c 0 <path>  |  ["termux-camera-photo", "-c", "0", "<path>"]
- termux-clipboard-set <text>  |  ["termux-clipboard-set", "<text>"]
```

```
- termux-tts-speak <text>  │  ["termux-tts-speak", "<text>"]
- am start -a android.intent.action.MAIN -c android.intent.category.LAUNCHER -p <package_
name>  │  ["am", "start", "-a", "android.intent.action.MAIN", "-c", "android.intent.categ
ory.LAUNCHER", "-p", "<package_name>"]"""

def ensure_commands_file():
    """Đ¡Đ¾Đ•Đ´Đ°ĐµÑ\202 Ñ\204Đ°Đ¹Đ» Ñ\201Đ¿Ñ\200Đ°Đ²Đ¾Ñ\207Đ½Đ¸Đ°Đ°, ĐµÑ\201Đ»Đ¸ ĐµĐ³Đ¾
Đ½ĐµÑ\202."""
    if not os.path.exists(COMMANDS_FILE):
        try:
            with open(COMMANDS_FILE, "w", encoding="utf-8") as f:
                f.write(DEFAULT_COMMANDS)
            print(f"[TESTER] Đ¡Đ¾Đ•Đ´Đ°Đ½ Ñ\201Đ¿Ñ\200Đ°Đ²Đ¾Ñ\207Đ½Đ¸Đ° Đ°Đ¾Đ¼Đ°Đ½Đ´: {CO
MMANDS_FILE}")
        except Exception as e:
            print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\201Đ¾Đ•Đ´Đ°Đ½Đ¸Đ¸ Ñ\201Đ¿Ñ
\200Đ°Đ²Đ¾Ñ\207Đ½Đ¸Đ°Đ°: {e}")

def load_commands():
    """Đ§Đ¸Ñ\202Đ°ĐµÑ\202 Ñ\201Đ¿Ñ\200Đ°Đ²Đ¾Ñ\207Đ½Đ¸Đ° Đ°Đ¾Đ¼Đ°Đ½Đ´ Đ¸Đ• Ñ\204Đ°Đ¹Đ»Đ° (
ĐµÑ\201Đ»Đ¸ ĐµÑ\201Ñ\202Ñ\214)."""
    if not os.path.exists(COMMANDS_FILE):
        return ""
    try:
        with open(COMMANDS_FILE, "r", encoding="utf-8") as f:
            return f.read().strip()
    except Exception as e:
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\207Ñ\202ĐµĐ½Đ¸Đ¸ Ñ\201Đ¿Ñ\200Đ°Đ²
Đ¾Ñ\207Đ½Đ¸Đ°Đ°: {e}")
        return ""

def search_docs(query):
    """Đ\230Ñ\211ĐµÑ\202 Ñ\201Ñ\202Ñ\200Đ¾Đ°Đ¸ Ñ\201 query Đ² termux_api.md (Đ½Đµ Đ±Đ¾Đ»Đ
µĐµ 10 Ñ\201Ñ\202Ñ\200Đ¾Đ°)."""
    if not os.path.exists(DOCS_FILE):
        return ""
    try:
        with open(DOCS_FILE, "r", encoding="utf-8") as f:
            lines = f.readlines()

        found = []
        for line in lines:
            if query.lower() in line.lower():
                found.append(line.strip())
                if len(found) >= 10:
                    break

        return "\n".join(found)
    except Exception as e:
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Đ¿Đ¾Đ¸Ñ\201Đ°Đµ Đ² Đ´Đ¾Đ°Ñ\203Đ¼ĐµĐ
½Ñ\202Đ°Ñ\206Đ¸Đ¸: {e}")
        return ""

def main():
    print("[TESTER] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Đ¡Đ¸Đ½Ñ\202Đ°Đ°Ñ\201Đ¸Ñ\207ĐµÑ\201Đ°Đ¾Đ³Đ¾ Đ
\234Đ¾Đ½Ñ\201Ñ\202Ñ\200Đ°...")

    if not os.path.exists(FACTORY_DIR):
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Đ¿Đ°Đ¿Đ°Đ° Ñ\204Đ°Đ±Ñ\200Đ¸Đ°, Đ½Đµ Đ½Đ°Đ¹Đ
´ĐµĐ½Đ°: {FACTORY_DIR}")
        return

    ensure_commands_file()

    if not os.path.exists(DRAFT_FILE):
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ñ\204Đ°Đ¹Đ» draft.py Đ½Đµ Đ½Đ°Đ¹Đ´ĐµĐ½: {DRA
FT_FILE}")
        return

    try:
```

```
        with open(DRAFT_FILE, "r", encoding="utf-8") as f:
            code_text = f.read().strip()
        if not code_text:
            print("[TESTER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ñ\204Ð°Ð¹Ð» draft.py Ð¿Ñ\203Ñ\201Ñ\202.")
            return
        print(f"[TESTER] Ð\232Ð¾Ð´ Ð¿Ñ\200Ð¾Ñ\207Ð¸Ñ\202Ð°Ð½ ({len(code_text)} Ñ\201Ð¸Ð¼Ð²Ð¾Ð»Ð²).")
    except Exception as e:
        print(f"[TESTER] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ñ\207Ñ\202ÐµÐ½Ð¸Ð¸ draft.py: {e}")
        return

    commands_text = load_commands()

    termux_commands = re.findall(r'termux-[a-z0-9-]+', code_text.lower())
    docs_context = ""
    for cmd in set(termux_commands):
        found = search_docs(cmd)
        if found:
            docs_context += f"\n[Ð\224Ð¾Ð°Ñ\203Ð¼ÐµÐ½Ñ\202Ð°Ñ\206Ð¸Ñ\217 Ð¿Ð¾ {cmd}]:\n{found}\n"

    prompt_parts = [
        "Ð¢Ñ\213 â\200\224 Ð¼Ð¾Ð´Ñ\203Ð»Ñ\214 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð¸. Ð¢Ð²Ð¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202 Ð±Ñ\203Ð´ÐµÑ\202 Ð°Ð²Ñ\202Ð¾Ð¼Ð°Ñ\202Ð¸Ñ\207ÐµÑ\201Ð°Ð¸ Ð¾Ð±Ñ\200ÐµÐ•Ð°Ð½ Ð¿Ð°Ñ\200Ñ\201ÐµÑ\200Ð¾Ð¼, ÐµÑ\201Ð»Ð¸ Ð¿Ñ\200ÐµÐ²Ñ\213Ñ\201Ð¸Ñ\202 Ð»Ð¸Ð¼Ð¸Ñ\202.",
        "Ð¤Ð\236Ð Ð\234Ð\220Ð¢ Ð\236Ð¢Ð\222Ð\225Ð¢Ð\220 Ð¡Ð¢Ð Ð\236Ð\223Ð\236:",
        "[Ð¡Ð¢Ð\220Ð¢Ð£Ð¡] | [Ð\237Ð Ð\230Ð§Ð\230Ð\235Ð\220]",
        "Ð\223Ð´Ðµ [Ð¡Ð¢Ð\220Ð¢Ð£Ð¡] â\200\224 Ñ\215Ñ\202Ð¾ Ñ\202Ð¾Ð»Ñ\214Ð°Ð¾ Ñ\201Ð»Ð¾Ð²Ð¾ Ð£Ð¡Ð¿Ð\237Ð\225Ð¥ Ð¸Ð»Ð¸ Ð\236Ð¨Ð\230Ð\221Ð\232Ð\220.",
        "Ð\223Ð´Ðµ [Ð\237Ð Ð\230Ð§Ð\230Ð\235Ð\220] â\200\224 Ð½Ðµ Ð±Ð¾Ð»ÐµÐµ 5 Ñ\201Ð»Ð¾Ð².",
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ñ\205Ð¾Ñ\200Ð¾Ñ\210ÐµÐ³Ð¾ Ð¾Ñ\202Ð²ÐµÑ\202Ð°: Ð£Ð¡Ð\237Ð\225Ð¥ | Ð°Ð¾Ð´ Ð°Ð¾Ñ\200Ñ\200ÐµÐ°Ñ\202ÐµÐ½",
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ð¿Ð»Ð¾Ñ\205Ð¾Ð³Ð¾ Ð¾Ñ\202Ð²ÐµÑ\202Ð°: Ð\236Ð¨Ð\230Ð\221Ð\232Ð\220 | Ð½ÐµÐ²ÐµÑ\200Ð½Ñ\213Ð¹ Ð¸Ð¼Ð¿Ð¾Ñ\200Ñ\202",
        "Ð\225Ñ\201Ð»Ð¸ Ñ\202Ð²Ð¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202 Ð±Ñ\203Ð´ÐµÑ\202 Ð´Ð»Ð¸Ð½Ð½ÐµÐµ, Ñ\201Ð¸Ñ\201Ñ\202ÐµÐ¼Ð° Ð²Ñ\213Ð´Ð°Ñ\201Ñ\202 Ð¾Ñ\210Ð¸Ð±Ð°Ñ\203 Ð¸ Ñ\202Ñ\213 Ð½Ðµ Ð¿Ñ\200Ð¾Ð¹Ð´ÐµÑ\210Ñ\214 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ñ\203. Ð\235Ðµ Ð¿Ð¸Ñ\210Ð¸ Ð½Ð¸Ñ\207ÐµÐ³Ð¾ Ð°Ñ\200Ð¾Ð¼Ð½Ðµ Ñ\215Ñ\202Ð¾Ð¹ Ñ\201Ñ\202Ñ\200Ð¾Ð°Ð¸."
    ]

    if commands_text:
        prompt_parts.append(
            "Ð¡Ð\237Ð Ð\220Ð\222Ð\236Ð§Ð\235Ð\230Ð\232 Ð\232Ð\236Ð\234Ð\220Ð\235Ð\224 TERMUX:\n" + commands_text +
            "\n\nÐ¡Ð²ÐµÑ\200Ñ\217Ð¹ Ð°Ð¾Ð¼Ð°Ð½Ð´Ñ\213 Ð² Ð°Ð¾Ð´Ðµ Ñ\201Ð¾ Ñ\201Ð¿Ñ\200Ð°Ð²Ð¾Ñ\207Ð½Ð°Ð¾Ð¼. "
            "Ð\225Ñ\201Ð»Ð¸ Ð°Ð¾Ð¼Ð°Ð½Ð´Ñ\213 Ð½ÐµÑ\202 Ð² Ñ\201Ð¿Ñ\200Ð°Ð²Ð¾Ñ\207Ð½Ð¸Ðµ â\200\224 Ñ\215Ñ\202Ð¾ Ð\236Ð¨Ð\230Ð\221Ð\232Ð\220."
        )

    if docs_context:
        prompt_parts.append(
            "Ð\222Ð¾Ñ\202 Ð²Ñ\213Ð´ÐµÑ\200Ð¶Ð°Ð¸ Ð¸Ð• Ð¾Ñ\204Ð¸Ñ\206Ð¸Ð°Ð»Ñ\214Ð½Ð¾Ð¹ Ð´Ð¾Ñ\203Ð¼ÐµÐ½Ñ\202Ð°Ñ\206Ð¸Ð¸ Termux:\n" +
            docs_context
        )

    prompt_parts.append(f"Ð\232Ð¾Ð´ Ð´Ð»Ñ\217 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð¸:\n{code_text}")
    prompt = "\n".join(prompt_parts)

    print(f"[TESTER] Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð°Ð° Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð° Ð² Ollama ({MODEL})...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True, text=True, timeout=600
        )
    except subprocess.TimeoutExpired:
        print("[TESTER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð½Ðµ Ð¾Ñ\202Ð²ÐµÑ\202Ð¸Ð»Ð° Ð•Ð° 10 Ð¼
```

```
Đ¸Đ½Ñ\203Ñ\202 (Ñ\202Đ°Đ¹Đ¼Đ°Ñ\203Ñ\202).")
        return
    except Exception as e:
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Đ²Ñ\213Đ•Đ¾Đ²Đµ Ollama: {e}")
        return

    if result.returncode != 0:
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ollama Đ²ĐµÑ\200Đ½Ñ\203Đ»Đ° Đ°Đ¾Đ´ {result.r
eturncode}.")
        print(f"[TESTER] stderr: {result.stderr}")
        return

    response_text = result.stdout.strip()[:60]
    if not response_text:
        print("[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ollama Đ²ĐµÑ\200Đ½Ñ\203Đ»Đ° Đ¿Ñ\203Ñ\201Ñ\202
Đ¾Đ¹ Đ¾Ñ\202Đ²ĐµÑ\202.")
        return

    try:
        with open(REPORT_FILE, "w", encoding="utf-8") as f:
            f.write(response_text)
        print(f"[TESTER] Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Đ°Đ° Đ•Đ°Đ²ĐµÑ\200Ñ\210ĐµĐ½Đ°. Đ ĐµĐ•Ñ\203Đ
»Ñ\214Ñ\202Đ°Ñ\202 Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½ Đ² report.txt")
        print(f"[TESTER] Đ\236Ñ\202Ñ\207ĐµÑ\202:\n{response_text}")
    except Exception as e:
        print(f"[TESTER] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½Đ¸Đ¸ Đ¾Ñ
\202Ñ\207ĐµÑ\202Đ°: {e}")
        return

if __name__ == "__main__":
    main()
-e


=== FILE: ./synthesizer.py ===
import os
import subprocess
import re

# --- Đ\235Đ\220Đ¡Đ¢Đ Đ\236Đ\231Đ\232Đ\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_1 = os.path.join(FACTORY_DIR, "draft_1.py")
DRAFT_2 = os.path.join(FACTORY_DIR, "draft_2.py")
DRAFT_3 = os.path.join(FACTORY_DIR, "draft_3.py")
DRAFT_FINAL = os.path.join(FACTORY_DIR, "draft_final.py")
ANALYSIS_FILE = os.path.join(FACTORY_DIR, "analysis.txt")
MODEL = "qwen2.5-coder:3b"

def clean_code(text):
    """Đ\236Ñ\207Đ¸Ñ\211Đ°ĐµÑ\202 Đ¾Ñ\202Đ²ĐµÑ\202 Đ¼Đ¾Đ´ĐµĐ»Đ¸ Đ¾Ñ\202 Đ¼Đ°Ñ\200Đ°Đ´Đ°Ñ
\203Đ½Đ°."""
    match = re.search(r'```(?:python)?\s*(.*?)```', text, re.DOTALL)
    if match:
        return match.group(1).strip()
    return text.strip()

def read_file(filepath):
    """Đ§Đ¸Ñ\202Đ°ĐµÑ\202 Ñ\204Đ°Đ¹Đ», ĐµÑ\201Đ»Đ¸ Đ¾Đ½ Ñ\201Ñ\203Ñ\211ĐµÑ\201Ñ\202Đ²Ñ
\203ĐµÑ\202."""
    if not os.path.exists(filepath):
        return None
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read().strip()
        return content if content else None
    except Exception as e:
        print(f"[SYNTH] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\207Ñ\202ĐµĐ½Đ¸Đ¸ {filepath}: {e}")
        return None

def main():
```

```
    print("[SYNTH] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ð\222ÐµÑ\200Ñ\205Ð¾Ð²Ð½Ð¾Ð³Ð¾ Ð¡Ð¸Ð½Ñ\202ÐµÐ•Ð°Ñ
\202Ð¾Ñ\200Ð°...")

    # 1. Ð§Ð¸Ñ\202Ð°ÐµÐ¼ analysis.txt
    analysis_text = read_file(ANALYSIS_FILE)
    if not analysis_text:
        print("[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: analysis.txt Ð½Ðµ Ð½Ð°Ð¹Ð´ÐµÐ½ Ð¸Ð»Ð¸ Ð¿Ñ\203Ñ
\201Ñ\202. Ð\230Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ñ\203Ñ\216 draft_1.py.")
        analysis_text = "1 | Ð\236Ð¨Ð\230Ð\221Ð\236Ð\232 Ð\235Ð\225Ð¢"

    print(f"[SYNTH] Ð\220Ð½Ð°Ð»Ð¸Ð•: {analysis_text[:200]}...")

    # 2. Ð\237Ð°Ñ\200Ñ\201Ð¸Ñ Ð½Ð¾Ð¼Ð¼ÐµÑ\200 Ð´Ñ\200Ð°Ñ\204Ñ\202Ð° Ð¸Ð• analysis.txt
    match = re.search(r'(\d)', analysis_text)
    if match:
        draft_number = match.group(1)
    else:
        draft_number = "1"

    print(f"[SYNTH] Ð\222Ñ\213Ð±Ñ\200Ð°Ð½ Ð´Ñ\200Ð°Ñ\204Ñ\202 â\204\226{draft_number}")

    # 3. Ð§Ð¸Ñ\202Ð°ÐµÐ¼ Ð\222Ð«Ð\221Ð Ð\220Ð\235Ð\235Ð«Ð\231 Ð´Ñ\200Ð°Ñ\204Ñ\202
    draft_map = {"1": DRAFT_1, "2": DRAFT_2, "3": DRAFT_3}
    selected_draft_path = draft_map.get(draft_number, DRAFT_1)
    draft_content = read_file(selected_draft_path)

    if not draft_content:
        print(f"[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð²Ñ\213Ð±Ñ\200Ð°Ð½Ð½Ñ\213Ð¹ Ð´Ñ\200Ð°Ñ\204Ñ
\202 {selected_draft_path} Ð¿Ñ\203Ñ\201Ñ\202. Ð\237Ñ\200Ð¾Ð±Ñ\203Ñ\216 draft_1.py.")
        draft_content = read_file(DRAFT_1)

    if not draft_content:
        print("[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð½ÐµÑ\202 Ð´Ð¾Ñ\201Ñ\202Ñ\203Ð¿Ð½Ñ\213Ñ\205 Ð´
Ñ\200Ð°Ñ\204Ñ\202Ð¾Ð².")
        return

    # 4. Ð\230Ð•Ð²Ð»ÐµÐ°Ð°ÐµÐ¼ Ñ\201Ð¿Ð¸Ñ\201Ð¾Ðº Ð¾Ñ\210Ð¸Ð±Ð¾Ðº Ð¸Ð• analysis.txt
    errors_match = re.search(r'Ð\236Ð¨Ð\230Ð\221Ð\232Ð\230:\s*(.+)', analysis_text, re.DO
TALL)
    if errors_match:
        errors_text = errors_match.group(1).strip()
    else:
        errors_text = "Ð\236Ð¨Ð\230Ð\221Ð\236Ð\232 Ð\235Ð\225Ð¢"

    print(f"[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð¸: {errors_text[:200]}...")

    # 5. Ð¤Ð¾Ñ\200Ð¼Ð¸Ñ\200Ñ\203ÐµÐ¼ Ð¿Ñ\200Ð¾Ð¼Ð¿Ñ\202 Ð´Ð»Ñ\217 Ð¡Ð¸Ð½Ñ\202ÐµÐ•Ð°Ñ\202Ð
¾Ñ\200Ð°
    prompt = (
        "Ð¢Ñ\213 â\200\224 Ð\223Ð»Ð°Ð²Ð½Ñ\213Ð¹ Ð\220Ñ\200Ñ\205Ð¸Ñ\202ÐµÐ°Ñ\202Ð¾Ñ\200. "
        f"Ð\222Ð¾Ñ\202 Ð²Ñ\213Ð±Ñ\200Ð°Ð½Ð½Ñ\213Ð¹ Ð»Ñ\203Ñ\207Ñ\210Ð¸Ð¹ Ð²Ð°Ñ\200Ð¸Ð°Ð½Ñ
\202 Ð°Ð¾Ð´Ð° (Ð´Ñ\200Ð°Ñ\204Ñ\202 â\204\226{draft_number}). "
        f"Ð\222 Ð½ÐµÐ¼ ÐµÑ\201Ñ\202Ñ\214 Ñ\201Ð»ÐµÐ´Ñ\203Ñ\216Ñ\211Ð¸Ðµ Ð¾Ñ\210Ð¸Ð±Ð°Ð¸ (
Ð¾Ñ\202 Ð\220Ð½Ð°Ð»Ð¸Ð•Ð°Ñ\202Ð¾Ñ\200Ð°): {errors_text}. "
        "Ð¢Ð²Ð¾Ñ\217 Ð•Ð°Ð´Ð°Ñ\207Ð° â\200\224 Ð¸Ñ\201Ð¿Ñ\200Ð°Ð²Ð¸Ñ\202Ñ\214 Ñ\215Ñ\202Ð
¸ Ð¾Ñ\210Ð¸Ð±Ð¾Ð¸, Ð´Ð¾Ð±Ð°Ð²Ð¸Ñ\202Ñ\214 Ð¾Ð±Ñ\200Ð°Ð±Ð¾Ñ\202Ð°Ñ\203 Ð¾Ñ\210Ð¸Ð±Ð¾Ð° Ð¸
Ñ\201Ð´ÐµÐ»Ð°Ñ\202Ñ\214 Ð°Ð¾Ð´ Ð¸Ð´ÐµÐ°Ð»Ñ\214Ð½Ñ\213Ð¼. "
        "Ð\235Ð\225 Ð¡Ð\234Ð\225Ð¨Ð\230Ð\222Ð\220Ð\231 ÐµÐ³Ð¾ Ñ\201 Ð´Ñ\200Ñ\203Ð³Ð¸Ð¼Ð¸
Ð²Ð°Ñ\200Ð¸Ð°Ð½Ñ\202Ð°Ð¼Ð¸. "
        "Ð\235Ð°Ð¿Ð¸Ñ\210Ð¸ Ñ\202Ð¾Ð»Ñ\214Ð°Ð¾ Ñ\204Ð¸Ð½Ð°Ð»Ñ\214Ð½Ñ\213Ð¹ Ð°Ð¾Ð´, Ð±ÐµÐ•
 Ð¾Ð±Ñ\212Ñ\217Ñ\201Ð½ÐµÐ½Ð¸Ð¹. "
        "Ð\230Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ñ\203Ð¹ Ñ\202Ð¾Ð»Ñ\214Ð°Ð¾ Ñ\201Ñ\202Ð°Ð½Ð´Ð°Ñ\200Ñ\202Ð½
Ñ\213Ðµ Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð¸. Python (os, subprocess, time, datetime, re, sys). "
        f"\n\n--- Ð\222Ð«Ð\221Ð Ð\220Ð\235Ð\235Ð«Ð\231 Ð\224Ð Ð\220Ð¤Ð¢ ---\n{draft_conte
nt}"
    )

    # 6. Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð»Ñ\217ÐµÐ¼ Ð² Ollama
    print(f"[SYNTH] Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð°Ð° Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð° Ð² Ollama ({MODEL})...
")
```

```
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True,
            text=True,
            timeout=1200
        )
    except subprocess.TimeoutExpired:
        print("[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð½Ðµ Ð¾Ñ\202Ð²ÐµÑ\202Ð¸Ð»Ð° Ð•Ð° 20 Ð¼Ð
¸Ð½Ñ\203Ñ\202.")
        return
    except Exception as e:
        print(f"[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð²Ñ\213Ð•Ð¾Ð²Ðµ Ollama: {e}")
        return

    if result.returncode != 0:
        print(f"[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð²ÐµÑ\200Ð½Ñ\203Ð»Ð° Ð°Ð¾Ð´ {result.re
turncode}.")
        return

    response_text = result.stdout.strip()
    if not response_text:
        print("[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð²ÐµÑ\200Ð½Ñ\203Ð»Ð° Ð¿Ñ\203Ñ\201Ñ\202Ð
¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202.")
        return

    # 7. Ð\236Ñ\207Ð¸Ñ\211Ð°ÐµÐ¼ Ð°Ð¾Ð´
    clean = clean_code(response_text)
    if not clean:
        print("[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð½Ðµ Ñ\203Ð´Ð°Ð»Ð¾Ñ\201Ñ\214 Ð¸Ð•Ð²Ð»ÐµÑ\207Ñ
\214 Ð°Ð¾Ð´.")
        return

    # 8. Ð¡Ð¾Ñ\205Ñ\200Ð°Ð½Ñ\217ÐµÐ¼
    try:
        with open(DRAFT_FINAL, "w", encoding="utf-8") as f:
            f.write(clean)
        print(f"[SYNTH] Ð¤Ð¸Ð½Ð°Ð»Ñ\214Ð½Ñ\213Ð¹ Ð°Ð¾Ð´ Ñ\201Ð¾Ñ\205Ñ\200Ð°Ð½Ñ\91ÐµÐ½ Ð² draf
t_final.py ({len(clean)} Ñ\201Ð¸Ð¼Ð²Ð¾Ð»Ð¾Ð²).")
    except Exception as e:
        print(f"[SYNTH] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ñ\201Ð¾Ñ\205Ñ\200Ð°Ð½Ñ\91ÐµÐ½Ð¸Ð¸: {e}")
        return

if __name__ == "__main__":
    main()
-e


=== FILE: ./director.py ===
import os
import subprocess
import shutil
import time
import datetime

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
SUCCESS_DIR = os.path.join(FACTORY_DIR, "success")
TRASH_DIR = os.path.join(FACTORY_DIR, "trash")
SANDBOX_DIR = os.path.join(FACTORY_DIR, "sandbox")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
FEEDBACK_FILE = os.path.join(FACTORY_DIR, "feedback.txt")
DRAFT_1 = os.path.join(FACTORY_DIR, "draft_1.py")
DRAFT_ANALYZED = os.path.join(FACTORY_DIR, "draft_analyzed.py")
DRAFT_FINAL = os.path.join(FACTORY_DIR, "draft_final.py")
SANDBOX_TARGET = os.path.join(SANDBOX_DIR, "voice_jarvis.py")
SANDBOX_SUCCESS_LOG = os.path.join(SANDBOX_DIR, "sandbox_success.log")
SANDBOX_ERROR_LOG = os.path.join(SANDBOX_DIR, "sandbox_error.log")

MAX_ATTEMPTS = 3
MODELS = {
```

```
    "coder": "qwen2.5-coder:3b", "analyzer": "qwen2.5-coder:3b",
    "logic": "llama3.2:1b", "syntax": "qwen2.5:1.5b",
    "critic_coder": "stable-code:3b", "judge": "qwen2.5-coder:3b"
}

def run_script(script_name):
    path = os.path.join(FACTORY_DIR, script_name)
    try:
        res = subprocess.run(["python", path], cwd=FACTORY_DIR, capture_output=True, text
=True, timeout=1200)
        return res.returncode == 0
    except Exception:
        return False

def stop_model(model_name):
    subprocess.run(["ollama", "stop", model_name], capture_output=True)

def main():
    for d in [SUCCESS_DIR, TRASH_DIR, SANDBOX_DIR]:
        os.makedirs(d, exist_ok=True)

    if not os.path.exists(TASK_FILE):
        print("[DIRECTOR] Đ\235ĐµÑ\202 task.txt")
        return

    for attempt in range(1, MAX_ATTEMPTS + 1):
        print(f"\n[DIRECTOR] === Đ\237Đ¾Đ¿Ñ\213Ñ\202Đ°Đ° {attempt} Đ¸Đ• {MAX_ATTEMPTS} ==
=")

        if os.path.exists(FEEDBACK_FILE): os.remove(FEEDBACK_FILE)

        # 1. Đ\232Đ¾Đ´ĐµÑ\200
        print("[DIRECTOR] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Đ\232Đ¾Đ´ĐµÑ\200Đ°...")
        if not run_script("coder.py"): continue
        stop_model(MODELS["coder"]); time.sleep(3)

        # 2. Đ\220Đ½Đ°Đ»Đ¸Ñ\202Đ¸Đ°
        print("[DIRECTOR] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Đ\220Đ½Đ°Đ»Đ¸Ñ\202Đ¸Đ°Đ°...")
        if not run_script("analyzer.py"): continue
        stop_model(MODELS["analyzer"]); time.sleep(3)

        # 3. Đ\232Ñ\200Đ¸Ñ\202Đ¸Đ°Đ¸
        run_script("critic_logic.py"); stop_model(MODELS["logic"]); time.sleep(3)
        run_script("critic_syntax.py"); stop_model(MODELS["syntax"]); time.sleep(3)
        run_script("critic_coder.py"); stop_model(MODELS["critic_coder"]); time.sleep(3)

        # 4. Đ¡Ñ\203Đ´Ñ\214Ñ\217
        print("[DIRECTOR] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Đ¡Ñ\203Đ´Ñ\214Đ¸...")
        if not run_script("judge.py"): continue
        stop_model(MODELS["judge"]); time.sleep(3)

        if not os.path.exists(DRAFT_FINAL): continue

        # 5. Đ\237ĐµÑ\201Đ¾Ñ\207Đ½Đ¸Ñ\206Đ°
        print("[DIRECTOR] Đ\227Đ°Đ¿Ñ\203Ñ\201Đ° Đ\237ĐµÑ\201Đ¾Ñ\207Đ½Đ¸Ñ\206Ñ\213...")
        shutil.copy2(DRAFT_FINAL, SANDBOX_TARGET)
        if not run_script("sandbox.py"): continue

        if os.path.exists(SANDBOX_SUCCESS_LOG):
            timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
            shutil.copy2(DRAFT_FINAL, os.path.join(SUCCESS_DIR, f"success_{timestamp}.py"
))
            print(f"\n[DIRECTOR] Đ£Đ¡Đ\237Đ\225Đ¥! Đ\232Đ¾Đ´ Đ² success_{timestamp}.py")
            return
        else:
            print("[DIRECTOR] Đ\237ĐµÑ\201Đ¾Ñ\207Đ½Đ¸Ñ\206Đ° Đ½Đ°Ñ\210Đ»Đ° Đ¾Ñ\210Đ¸Đ±Đ°Đ
¸. Đ\227Đ°Đ¿Đ¸Ñ\201Ñ\213Đ²Đ°Ñ\216 Ñ\204Đ¸Đ´Đ±ĐµĐ°...")
            shutil.copy2(DRAFT_FINAL, os.path.join(TRASH_DIR, f"trash_{attempt}.py"))

            # Đ\237Đ¸Ñ\210ĐµĐ¼ Đ¾Ñ\210Đ¸Đ±Đ°Ñ\203 Đ\222 Đ\236Đ¢Đ\224Đ\225Đ\233Đ¬Đ\235Đ«Đ
```

```
\231 Ð¤Ð\220Ð\231Ð\233, Ð½Ðµ Ñ\202Ñ\200Ð¾Ð³Ð°Ñ\217 task.txt
            fb = []
            for err_file in ["logic_errors.txt", "syntax_errors.txt", "coder_errors.txt"]
:
                p = os.path.join(FACTORY_DIR, err_file)
                if os.path.exists(p):
                    with open(p, "r", encoding="utf-8") as f:
                        content = f.read().strip()
                        if content and "Ð£Ð¡Ð\237Ð\225Ð¥" not in content.upper():
                            fb.append(f"[{err_file}]: {content}")

            if os.path.exists(SANDBOX_ERROR_LOG):
                with open(SANDBOX_ERROR_LOG, "r", encoding="utf-8") as f:
                    fb.append(f"[Ð\237ÐµÑ\201Ð¾Ñ\207Ð½Ð¸Ñ\206Ð°]: {f.read().strip()}")

            if fb:
                with open(FEEDBACK_FILE, "w", encoding="utf-8") as f:
                    f.write("\n".join(fb))

    print(f"\n[DIRECTOR] Ð\235Ðµ Ñ\203Ð´Ð°Ð»Ð¾Ñ\201Ñ\214 Ð•Ð° {MAX_ATTEMPTS} Ð¿Ð¾Ð¿Ñ\213Ñ
\202Ð°Ð¸.")

if __name__ == "__main__":
    main()
-e


=== FILE: ./final_code.py ===
import subprocess

def toggle_torch():
    try:
        subprocess.run(["termux-torch"], check=True)
    except subprocess.CalledProcessError:
        subprocess.run(["termux-torch", "off"], check=True)

# Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ð¸Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ð¾Ð²Ð°Ð½Ð¸Ñ\217 Ñ\204Ñ\203Ð½Ð°Ñ\206Ð¸Ð¸
toggle_torch()-e


=== FILE: ./termux_commands.txt ===
- termux-torch on  |  ["termux-torch", "on"]
- termux-torch off  |  ["termux-torch", "off"]
- termux-brightness <0-255>  |  ["termux-brightness", "<0-255>"]
- termux-volume music <0-15>  |  ["termux-volume", "music", "<0-15>"]
- termux-camera-photo -c 0 <path>  |  ["termux-camera-photo", "-c", "0", "<path>"]
- termux-clipboard-set <text>  |  ["termux-clipboard-set", "<text>"]
- termux-tts-speak <text>  |  ["termux-tts-speak", "<text>"]
- am start -a android.intent.action.MAIN -c android.intent.category.LAUNCHER -p <package_
name>  |  ["am", "start", "-a", "android.intent.action.MAIN", "-c", "android.intent.categ
ory.LAUNCHER", "-p", "<package_name>"]-e


=== FILE: ./logician.py ===
import os
import subprocess
import re

# --- Ð\235Ð\220Ð¡Ð¢Ð Ð\236Ð\231Ð\232Ð\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft.py")
LOGIC_REPORT_FILE = os.path.join(FACTORY_DIR, "logic_report.txt")
DOCS_FILE = os.path.join(FACTORY_DIR, "docs", "termux_api.md")
MODEL = "llama3.2:1b"

def search_docs(query):
    """Ð\230Ñ\211ÐµÑ\202 Ñ\201Ñ\202Ñ\200Ð¾Ð°Ð¸ Ñ\201 query Ð² termux_api.md (Ð½Ðµ Ð±Ð¾Ð»Ð
µÐµ 10 Ñ\201Ñ\202Ñ\200Ð¾Ð°)."""
    if not os.path.exists(DOCS_FILE):
        return ""
```

```
    try:
        with open(DOCS_FILE, "r", encoding="utf-8") as f:
            lines = f.readlines()

        found = []
        for line in lines:
            if query.lower() in line.lower():
                found.append(line.strip())
                if len(found) >= 10:
                    break

        return "\n".join(found)
    except Exception as e:
        print(f"[LOGICIAN] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð¿Ð¾Ð¸Ñ\201Ð°Ðµ Ð² Ð´Ð¾Ñ\203Ð¼Ðµ
½Ñ\202Ð°Ñ\206Ð¸Ð¸: {e}")
        return ""

def main():
    print("[LOGICIAN] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ð\233Ð¾Ð³Ð¸Ð°Ð°...")

    if not os.path.exists(FACTORY_DIR):
        print(f"[LOGICIAN] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¿Ð°Ð¿Ð° Ñ\204Ð°Ð±Ñ\200Ð¸Ð°Ð¸ Ð½Ðµ Ð½Ð°Ð
¹Ð´ÐµÐ½Ð°: {FACTORY_DIR}")
        return

    if not os.path.exists(DRAFT_FILE):
        print(f"[LOGICIAN] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ñ\204Ð°Ð¹Ð» draft.py Ð½Ðµ Ð½Ð°Ð¹Ð´ÐµÐ½: {D
RAFT_FILE}")
        return

    try:
        with open(DRAFT_FILE, "r", encoding="utf-8") as f:
            code_text = f.read().strip()
        if not code_text:
            print("[LOGICIAN] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ñ\204Ð°Ð¹Ð» draft.py Ð¿Ñ\203Ñ\201Ñ\202.
")
            return
        print(f"[LOGICIAN] Ð\232Ð¾Ð´ Ð¿Ñ\200Ð¾Ñ\207Ð¸Ñ\202Ð°Ð½ ({len(code_text)} Ñ\201Ð¸Ð
¼Ð²Ð¾Ð»Ð¾Ð²).")
    except Exception as e:
        print(f"[LOGICIAN] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ñ\207Ñ\202ÐµÐ½Ð¸Ð¸ draft.py: {e}"
)
        return

    termux_commands = re.findall(r'termux-[a-z0-9-]+', code_text.lower())
    docs_context = ""
    for cmd in set(termux_commands):
        found = search_docs(cmd)
        if found:
            docs_context += f"\n[Ð\224Ð¾Ð°Ñ\203Ð¼ÐµÐ½Ñ\202Ð°Ñ\206Ð¸Ñ\217 Ð¿Ð¾ {cmd}]:\n{f
ound}\n"

    prompt_parts = [
        "Ð¢Ñ\213 â\200\224 Ð¼Ð¾Ð´Ñ\203Ð»Ñ\214 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð¸. Ð¢Ð²Ð¾Ð¹ Ð¾Ñ\202Ð²Ðµ
Ñ\202 Ð±Ñ\203Ð´ÐµÑ\202 Ð°Ð²Ñ\202Ð¾Ð¼Ð°Ñ\202Ð¸Ñ\207ÐµÑ\201Ð°Ð¸ Ð¾Ð±Ñ\200ÐµÐ•Ð°Ð½ Ð¿Ð°Ñ
\200Ñ\201ÐµÑ\200Ð¾Ð¼, ÐµÑ\201Ð»Ð¸ Ð¿Ñ\200ÐµÐ²Ñ\213Ñ\201Ð¸Ñ\202 Ð»Ð¸Ð¼Ð¸Ñ\202.",
        "Ð¤Ð\236Ð Ð\234Ð\220Ð¢ Ð\236Ð¢Ð\222Ð\225Ð¢Ð\220 Ð¡Ð¢Ð Ð\236Ð\223Ð\236:",
        "[Ð¡Ð¢Ð\220Ð¢Ð£Ð¡] | [Ð\237Ð Ð\230ÐŞÐ\230Ð\235Ð\220]",
        "Ð\223Ð´Ðµ [Ð¡Ð¢Ð\220Ð¢Ð£Ð¡] â\200\224 Ñ\215Ñ\202Ð¾ Ñ\202Ð¾Ð»Ñ\214Ð°Ð¾ Ñ\201Ð»Ð¾Ð
²Ð¾ Ð£Ð¡Ð¿Ð\237Ð\225Ð¥ Ð¸Ð»Ð¸ Ð\236Ð¨Ð\230Ð\221Ð\232Ð\220.",
        "Ð\223Ð´Ðµ [Ð\237Ð Ð\230ÐŞÐ\230Ð\235Ð\220] â\200\224 Ð½Ðµ Ð±Ð¾Ð»ÐµÐµ 5 Ñ\201Ð»Ð¾Ð
².",
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ñ\205Ð¾Ñ\200Ð¾Ñ\210ÐµÐ³Ð¾ Ð¾Ñ\202Ð²ÐµÑ\202Ð°: Ð£Ð¡Ð¿Ð\237Ð
\225Ð¥ | Ð°Ð¾Ð´ Ð»Ð¾Ð³Ð¸Ñ\207ÐµÐ½",
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ð¿Ð»Ð¾Ñ\205Ð¾Ð³Ð¾ Ð¾Ñ\202Ð²ÐµÑ\202Ð°: Ð\236Ð¨Ð\230Ð\221Ð
\232Ð\220 | Ð¿Ð¾Ð²Ñ\202Ð¾Ñ\200 Ð´ÐµÐ¹Ñ\201Ñ\202Ð²Ð¸Ñ\217",
        "Ð\225Ñ\201Ð»Ð¸ Ñ\202Ð²Ð¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202 Ð±Ñ\203Ð´ÐµÑ\202 Ð´Ð»Ð¸Ð½Ð½ÐµÐµ, Ñ
\201Ð¸Ñ\201Ñ\202ÐµÐ¼Ð° Ð²Ñ\213Ð´Ð°Ñ\201Ñ\202 Ð¾Ñ\210Ð¸Ð±Ð°Ñ\203 Ð¸ Ñ\202Ñ\213 Ð½Ðµ Ð¿Ñ
\200Ð¾Ð¹Ð´ÐµÑ\210Ñ\214 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ñ\203. Ð\235Ðµ Ð¿Ð¸Ñ\210Ð¸ Ð½Ð¸Ñ\207ÐµÐ³Ð¾ Ð°Ñ
\200Ð¾Ð¼Ðµ Ñ\215Ñ\202Ð¾Ð¹ Ñ\201Ñ\202Ñ\200Ð¾Ð°Ð¸."
```

```
    ]

    if docs_context:
        prompt_parts.append(
            "Đ\222Đ¾Ñ\202 Đ²Ñ\213Đ´ĐµÑ\200Đ¶Đ°Đ¸ Đ¸Đ• Đ¾Ñ\204Đ¸Ñ\206Đ¸Đ°Đ»Ñ\214Đ½Đ¾Đ¹ Đ´Đ
¾Đ°Ñ\203Đ¼ĐµĐ½Ñ\202Đ°Ñ\206Đ¸Đ¸ Termux:\n" +
            docs_context +
            "\nĐ¡Đ²ĐµÑ\200Ñ\217Đ¹ Đ°Đ¾Đ´ Ñ\201 Ñ\215Ñ\202Đ¸Đ¼Đ¸ Đ²Ñ\213Đ´ĐµÑ\200Đ¶Đ°Đ°Đ¼Đ
¸."
        )

    prompt_parts.append(f"Đ\232Đ¾Đ´ Đ´Đ»Ñ\217 Đ¿Ñ\200Đ¾Đ²ĐµÑ\200Đ°Đ¸:\n{code_text}")
    prompt = "\n".join(prompt_parts)

    print(f"[LOGICIAN] Đ\236Ñ\202Đ¿Ñ\200Đ°Đ²Đ°Đ° Đ•Đ°Đ¿Ñ\200Đ¾Ñ\201Đ° Đ² Ollama ({MODEL})
...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True, text=True, timeout=600
        )
    except subprocess.TimeoutExpired:
        print("[LOGICIAN] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ollama Đ½Đµ Đ¾Ñ\202Đ²ĐµÑ\202Đ¸Đ»Đ° Đ•Đ° 10
Đ¼Đ¸Đ½Ñ\203Ñ\202 (Ñ\202Đ°Đ¹Đ¼Đ°Ñ\203Ñ\202).")
        return
    except Exception as e:
        print(f"[LOGICIAN] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Đ²Ñ\213Đ•Đ¾Đ²Đµ Ollama: {e}")
        return

    if result.returncode != 0:
        print(f"[LOGICIAN] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ollama Đ²ĐµÑ\200Đ½Ñ\203Đ»Đ° Đ°Đ¾Đ´ {result
.returncode}.")
        print(f"[LOGICIAN] stderr: {result.stderr}")
        return

    response_text = result.stdout.strip()[:60]
    if not response_text:
        print("[LOGICIAN] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: Ollama Đ²ĐµÑ\200Đ½Ñ\203Đ»Đ° Đ¿Ñ\203Ñ\201Ñ
\202Đ¾Đ¹ Đ¾Ñ\202Đ²ĐµÑ\202.")
        return

    try:
        with open(LOGIC_REPORT_FILE, "w", encoding="utf-8") as f:
            f.write(response_text)
        print(f"[LOGICIAN] Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Đ°Đ° Đ•Đ°Đ²ĐµÑ\200Ñ\210ĐµĐ½Đ°. Đ ĐµĐ•Ñ
\203Đ»Ñ\214Ñ\202Đ°Ñ\202 Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½ Đ² logic_report.txt")
        print(f"[LOGICIAN] Đ\236Ñ\202Ñ\207ĐµÑ\202:\n{response_text}")
    except Exception as e:
        print(f"[LOGICIAN] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½Đ¸Đ¸ Đ¾Ñ
\202Ñ\207ĐµÑ\202Đ°: {e}")
        return

if __name__ == "__main__":
    main()

-e


=== FILE: ./appeal.py ===
import os
import subprocess
import shutil

# --- Đ\235Đ\220Đ¡Đ¢Đ Đ\236Đ\231Đ\232Đ\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
APPEAL_DIR = os.path.join(FACTORY_DIR, "appeal")
SUCCESS_DIR = os.path.join(FACTORY_DIR, "success")
TRASH_DIR = os.path.join(FACTORY_DIR, "trash")
MODEL = "qwen2.5-coder:3b"
```

```
def main():
    print("[APPEAL] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ð\220Ð¿ÐµÐ»Ð»Ñ\217Ñ\206Ð¸Ð¾Ð½Ð½Ð¾Ð³Ð¾ Ñ\201Ñ
\203Ð´Ð°...")

    if not os.path.exists(APPEAL_DIR):
        print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¿Ð°Ð¿Ð° appeal/ Ð½Ðµ Ð½Ð°Ð¹Ð´ÐµÐ½Ð°.")
        return

    # Ð\221ÐµÑ\200ÐµÐ¼ Ð¿Ð¾Ñ\201Ð»ÐµÐ´Ð½Ð¸Ð¹ Ñ\204Ð°Ð¹Ð» Ð¸Ð• appeal/
    files = sorted(
        [f for f in os.listdir(APPEAL_DIR) if f.endswith(".py")],
        key=lambda x: os.path.getmtime(os.path.join(APPEAL_DIR, x))
    )

    if not files:
        print("[APPEAL] Ð\237Ð°Ð¿Ð°Ð° appeal/ Ð¿Ñ\203Ñ\201Ñ\202Ð°. Ð\235ÐµÑ\207ÐµÐ³Ð¾ Ñ
\200Ð°Ñ\201Ñ\201Ð¼Ð°Ñ\202Ñ\200Ð¸Ð²Ð°Ñ\202Ñ\214.")
        return

    target_file = os.path.join(APPEAL_DIR, files[-1])
    print(f"[APPEAL] Ð Ð°Ñ\201Ñ\201Ð¼Ð°Ñ\202Ñ\200Ð¸Ð²Ð°ÐµÑ\202Ñ\201Ñ\217 Ñ\204Ð°Ð¹Ð»: {ta
rget_file}")

    try:
        with open(target_file, "r", encoding="utf-8") as f:
            code_text = f.read().strip()
    except Exception as e:
        print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ñ\207Ñ\202ÐµÐ½Ð¸Ð¸ Ñ\204Ð°Ð¹Ð»Ð°: {
e}")
        return

    if not code_text:
        print("[APPEAL] Ð¤Ð°Ð¹Ð» Ð¿Ñ\203Ñ\201Ñ\202. Ð\237ÐµÑ\200ÐµÐ¼Ñ\210Ñ\203 Ð² trash
/.")
        shutil.move(target_file, os.path.join(TRASH_DIR, os.path.basename(target_file)))
        return

    # Ð\237Ñ\200Ð¾Ð¼Ð¿Ñ\202 Ð¡Ñ\203Ð´Ñ\214Ð¸
    prompt = (
        "Ð¢Ñ\213 â\200\224 Ð¡Ñ\203Ð´Ñ\214Ñ\217. Ð-Ñ\202Ð¾Ñ\202 Ð°Ð¾Ð´ Ð±Ñ\213Ð» Ð•Ð°Ð±Ñ
\200Ð°Ð°Ð¾Ð²Ð°Ð½ Ð´Ñ\200Ñ\203Ð³Ð¸Ð¼ Ð\230Ð\230. "
        "Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ñ\214 ÐµÐ³Ð¾ ÐµÑ\211Ðµ Ñ\200Ð°Ð•, Ð¸Ð³Ð½Ð¾Ñ\200Ð¸Ñ\200Ñ\203
Ñ\217 Ð¿Ñ\200ÐµÐ´Ñ\213Ð´Ñ\203Ñ\211ÐµÐµ Ð¼Ð½ÐµÐ½Ð¸Ðµ. "
        "Ð\225Ñ\201Ð»Ð¸ Ð°Ð¾Ð´ Ñ\205Ð¾Ñ\200Ð¾Ñ\210Ð¸Ð¹ â\200\224 Ð½Ð°Ð¿Ð¸Ñ\210Ð¸ Ð\236Ð
\237Ð Ð\220Ð\222Ð\224Ð\220Ð¢Ð¬. "
        "Ð\225Ñ\201Ð»Ð¸ Ð¿Ð»Ð¾Ñ\205Ð¾Ð¹ â\200\224 Ð½Ð°Ð¿Ð¸Ñ\210Ð¸ Ð\236Ð\221Ð\222Ð\230Ð
\235Ð\230Ð¢Ð¬.\n"
        f"Ð\232Ð¾Ð´:\n{code_text}"
    )

    print(f"[APPEAL] Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð°Ð° Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð° Ð¡Ñ\203Ð´Ñ\214Ðµ ({MO
DEL})...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True, text=True, timeout=600
        )
    except subprocess.TimeoutExpired:
        print("[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¡Ñ\203Ð´Ñ\214Ñ\217 Ð½Ðµ Ð¾Ñ\202Ð²ÐµÑ\202Ð¸Ð»
 Ð•Ð° 10 Ð¼Ð¸Ð½Ñ\203Ñ\202.")
        return
    except Exception as e:
        print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð²Ñ\213Ð•Ð¾Ð²Ðµ Ollama: {e}")
        return

    if result.returncode != 0:
        print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð²ÐµÑ\200Ð½Ñ\203Ð»Ð° Ð°Ð¾Ð´ {result.r
eturncode}.")
        return
```

```
    verdict = result.stdout.strip().upper()
    print(f"[APPEAL] Ð\222ÐµÑ\200Ð´Ð¸Ð°Ñ\202 Ð¡Ñ\203Ð´Ñ\214Ð¸:\n{result.stdout.strip()[:3
00]}...")

    if "Ð\236Ð\237Ð Ð\220Ð\222Ð\224Ð\220Ð¢Ð¬" in verdict:
        print("[APPEAL] Ð\222ÐµÑ\200Ð´Ð¸Ð°Ñ\202: Ð\236Ð\237Ð Ð\220Ð\222Ð\224Ð\220Ð¢Ð¬. Ð
\237ÐµÑ\200ÐµÐ½Ð¾Ñ\210Ñ\203 Ð² success/.")
        try:
            shutil.move(target_file, os.path.join(SUCCESS_DIR, os.path.basename(target_fi
le)))
        except Exception as e:
            print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð¿ÐµÑ\200ÐµÐ½Ð¾Ñ\201Ðµ Ð² succe
ss/: {e}")
    elif "Ð\236Ð\221Ð\222Ð\230Ð\235Ð\230Ð¢Ð¬" in verdict:
        print("[APPEAL] Ð\222ÐµÑ\200Ð´Ð¸Ð°Ñ\202: Ð\236Ð\221Ð\222Ð\230Ð\235Ð\230Ð¢Ð¬. Ð
\237ÐµÑ\200ÐµÐ½Ð¾Ñ\210Ñ\203 Ð² trash/.")
        try:
            shutil.move(target_file, os.path.join(TRASH_DIR, os.path.basename(target_file
)))
        except Exception as e:
            print(f"[APPEAL] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð¿ÐµÑ\200ÐµÐ½Ð¾Ñ\201Ðµ Ð² trash
/: {e}")
    else:
        print("[APPEAL] Ð\235ÐµÐ¸Ð•Ð²ÐµÑ\201Ñ\202Ð½Ñ\213Ð¹ Ð²ÐµÑ\200Ð´Ð¸Ð°Ñ\202. Ð\236Ñ
\201Ñ\202Ð°Ð²Ð»Ñ\217Ñ\216 Ð² appeal/.")

if __name__ == "__main__":
    main()
-e


=== FILE: ./update_docs.py ===
import os
import urllib.request

# --- Ð\235Ð\220Ð¡Ð¢Ð Ð\236Ð\231Ð\232Ð\230 ---
DOCS_DIR = os.path.expanduser("~/takcas_factory/docs")
MAX_SIZE = 8 * 1024 * 1024  # 8 Ð\234Ð\221

# WHITELIST: Ñ\202Ð¾Ð»Ñ\214Ð°Ð¼ Ñ\215Ñ\202Ð¸ Ñ\201Ñ\201Ñ\213Ð»Ð°Ð¸ Ð±Ñ\203Ð´Ñ\203Ñ\202 Ñ
\201Ð°Ð°Ñ\207Ð°Ð½Ñ\213 (Ð¼Ð°Ð°Ñ\201Ð¸Ð¼Ñ\203Ð¼ 3)
WHITELIST = [
    {
        "url": "https://raw.githubusercontent.com/termux/termux-api/master/README.md",
        "filename": "termux_api.md"
    },
    # Ð\224Ð¾Ð±Ð°Ð²Ñ\214 Ð´Ð¾ 2 Ð´Ñ\200Ñ\203Ð³Ð¸Ñ\205 Ñ\201Ñ\201Ñ\213Ð»Ð¾Ðº (ÐµÑ\201Ð»Ð¸
Ð½Ñ\203Ð¶Ð½Ð¾)
    # {
    #     "url": "https://example.com/some_doc.md",
    #     "filename": "some_doc.md"
    # },
]

def download_file(url, filepath):
    """
    Ð¡Ð°Ð°Ñ\207Ð¸Ð²Ð°ÐµÑ\202 Ñ\204Ð°Ð¹Ð» Ñ\201 Ð¿Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð¾Ð¹ Ñ\200Ð°Ð•Ð¼ÐµÑ
\200Ð°.
    Ð\222Ð¾Ð•Ð²Ñ\200Ð°Ñ\211Ð°ÐµÑ\202 True (Ñ\203Ñ\201Ð¿ÐµÑ\205) / False (Ð¾Ñ\210Ð¸Ð±Ð°Ð°)
.
    """
    try:
        # Ð\236Ñ\202Ð°Ñ\200Ñ\213Ð²Ð°ÐµÐ¼ URL (Ñ\201 Ñ\202Ð°Ð¹Ð¼Ð°Ñ\203Ñ\202Ð¾Ð¼ 30 Ñ\201Ð
µÐ°)
        req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
        with urllib.request.urlopen(req, timeout=30) as response:
            # Ð§Ð¸Ñ\202Ð°ÐµÐ¼ Ð´Ð°Ð½Ð½Ñ\213Ðµ (Ñ\201 Ð»Ð¸Ð¼Ð¸Ñ\202Ð¾Ð¼ Ñ\200Ð°Ð•Ð¼ÐµÑ\200
Ð°)
            data = response.read(MAX_SIZE + 1)  # Ð§Ð¸Ñ\202Ð°ÐµÐ¼ Ð½Ð° 1 Ð±Ð°Ð¹Ñ\202 Ð±Ð¾
Ð»Ñ\214Ñ\210Ðµ Ð»Ð¸Ð¸Ñ\202Ð°
```

```
            # Р\237Ñ\200Р¾Р²РµÑ\200Р°Р° Ñ\200Р°Р•Р¼РµÑ\200Р°
            if len(data) > MAX_SIZE:
                print(f"[DOCS] Р\236Ñ\210Р¸Р±Р°Р°: Р¤Р°Р¹Р» Ñ\201Р»Р¸Ñ\210Р°Р¾Р¼ Р±Р¾Р»Ñ
\214Ñ\210Р¾Р¹, Р¾Ñ\202Р°Р»Р¾Р½РµÐ½ (>{MAX_SIZE} Р±Р°Р¹Ñ\202)")
                return False

            # Р\227Р°Р¿Р¸Ñ\201Ñ\213Р²Р°РµÐ¼ Р² Ñ\204Р°Р¹Р»
            with open(filepath, "wb") as f:
                f.write(data)

            print(f"[DOCS] Р¡Р°Р°Ñ\207Р°Р½Р¾: {filepath} ({len(data)} Р±Р°Р¹Ñ\202)")
            return True

    except Exception as e:
        print(f"[DOCS] Р\236Ñ\210Р¸Р±Р°Р° Р¿Ñ\200Р¸ Ñ\201Р°Р°Ñ\207Р¸Р²Р°Р½Р¸Р¸ {url}: {e}
")
        return False

def main():
    print("[DOCS] Р\227Р°Р¿Ñ\203Ñ\201Р° Р¾Р±Р½Р¾Р²Р»РµÐ½Р¸Ñ\217 Р´Р¾Р°Ñ\203Ð¼ÐµÐ½Ñ\202Р°Ñ
\206Р¸Р¸...")

    # 1. Р¡Р¾Р•Р´Р°РµÐ¼ Р¿Р°Р¿Р°Ñ\203 docs, РµÑ\201Р»Р¸ РµÐµ Р½ÐµÑ\202
    try:
        os.makedirs(DOCS_DIR, exist_ok=True)
        print(f"[DOCS] Р\237Р°Р¿Р°Р° Ñ\201Р¾Р•Р´Р°Р½Р°: {DOCS_DIR}")
    except Exception as e:
        print(f"[DOCS] Р\236Ñ\210Р¸Р±Р°Р° Р¿Ñ\200Р¸ Ñ\201Р¾Р•Р´Р°Р½Р¸Р¸ Р¿Р°Р¿Р¸: {e}")
        return

    # 2. Р\237Ñ\200Р¾Р²РµÑ\200Ñ\217РµÐ¼ Р»Р¸Р¼Р¸Ñ\202 Ñ\201Ñ\201Ñ\213Р»Р¾Р°
    if len(WHITELIST) > 3:
        print("[DOCS] Р\236Ñ\210Р¸Р±Р°Р°: Р\222 WHITELIST Р±Р¾Р»Ñ\214Ñ\210Рµ 3 Ñ\201Ñ\201
Ñ\213Р»Р¾Р°. Р\230Ñ\201Р¿Ñ\200Р°Р²Ñ\214 Ñ\201Р¿Р¸Ñ\201Р¾Р°.")
        return

    # 3. Р\236Р±Ñ\200Р°Р±Р°Ñ\202Ñ\213Р²Р°РµÐ¼ Р°Р°Р¶Р´Ñ\203Ñ\216 Ñ\201Ñ\201Ñ\213Р»Р°Ñ\203
    for item in WHITELIST:
        url = item["url"]
        filename = item["filename"]
        filepath = os.path.join(DOCS_DIR, filename)

        print(f"\n[DOCS] Р\236Р±Ñ\200Р°Р±Р¾Ñ\202Р°Р°: {url}")

        # 3.1 Р\237Ñ\200Р¾Р²РµÑ\200Р°Р° Р½Р° Р¿Р¾Р²Ñ\202Р¾Ñ\200Р½Р¾Рµ Ñ\201Р°Р°Ñ\207Р¸Р²Р
°Р½Р¸Рµ
        if os.path.exists(filepath) and os.path.getsize(filepath) > 0:
            print(f"[DOCS] Р¤Р°Р¹Р» Ñ\203Р¶Рµ Ñ\201Ñ\203Ñ\211РµÑ\201Ñ\202Р²Ñ\203РµÑ\202:
{filepath} (Р¿Ñ\200Р¾Р¿Ñ\203Ñ\201Р°Р°РµÐ¼)")
            continue

        # 3.2 Р¡Р°Р°Ñ\207Р¸Р²Р°РµÐ¼
        download_file(url, filepath)

    print("\n[DOCS] Р\236Р±Ñ\200Р°Р²Р»РµÐ½Р¸Рµ Р´Р¾Р°Ñ\203Ð¼ÐµÐ½Ñ\202Р°Ñ\206Р¸Р¸ Р•Р°Р²РµÑ
\200Ñ\210РµÐ½Р¾.")

if __name__ == "__main__":
    main()
-e


=== FILE: ./success/appeal_3.py ===
def number_to_string(number):
    return f"{number:02}"-e


=== FILE: ./success/success_1.py ===
import datetime
```

```
with open('report.txt', 'w') as file:
    file.write(datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S'))-e


=== FILE: ./success/success_2026-10-25_13-34-21.py ===
import os
import random

# Ð¡Ð¾Ð·Ð´Ð°ÐµÐ¼ Ð¿Ð°Ð¿ÐºÑƒ 'backup_test'
if not os.path.exists('backup_test'):
    os.makedirs('backup_test')

# Ð\227Ð°Ð¿Ð¸Ñ\201Ñ\214 Ñ\201Ð»Ñ\203Ñ\207Ð°Ð¹Ð½Ñ\213Ñ\205 Ñ\207Ð¸Ñ\201ÐµÐ» Ð² 5 Ñ\202ÐµÐ°
Ñ\201Ñ\202Ð¾Ð²Ñ\213Ñ\205 Ñ\204Ð°Ð¹Ð»Ð¾Ð²
for i in range(1, 6):
    with open(f'backup_test/file_{i}.txt', 'w') as file:
        file.write(str(random.randint(1, 100)))

# Ð§Ñ\202ÐµÐ½Ð¸Ðµ Ð²Ñ\201ÐµÑ\205 Ñ\207Ð¸Ñ\201ÐµÐ» Ð¸Ð• Ñ\204Ð°Ð¹Ð»Ð¾Ð² Ð¸ Ð²Ñ\213Ñ\207Ð¸Ñ
\201Ð»ÐµÐ½Ð¸Ðµ Ñ\201Ñ\203Ð¼Ð¼Ñ\213
total_sum = 0
for i in range(1, 6):
    with open(f'backup_test/file_{i}.txt', 'r') as file:
        total_sum += int(file.read())

# Ð\227Ð°Ð¿Ð¸Ñ\201Ñ\214 Ñ\201Ñ\203Ð¼Ð¼Ñ\213 Ð² Ð½Ð¾Ð²Ñ\213Ð¹ Ñ\204Ð°Ð¹Ð» 'total_sum.txt'
with open('backup_test/total_sum.txt', 'w') as file:
    file.write(str(total_sum))-e


=== FILE: ./success/success_2026-10-25_13-40-49.py ===
import os
import random

def create_folder_and_files():
    # Ð¡Ð¾Ð·Ð´Ð°ÐµÐ¼ Ð¿Ð°Ð¿Ñ\203 'secure_data'
    if not os.path.exists('secure_data'):
        os.makedirs('secure_data')

    # Ð¡Ð¾Ð·Ð´Ð°ÐµÐ¼ 5 Ñ\204Ð°Ð¹Ð»Ð¾Ð² 'data_1.txt' - 'data_5.txt' Ð¸ Ð•Ð°Ð¿Ð¸Ñ\201Ñ\213Ð
²Ð°ÐµÐ¼ Ñ\201Ð»Ñ\203Ñ\207Ð°Ð¹Ð½Ñ\213Ðµ Ñ\207\033[1D\033[K
Ñ\207Ð¸Ñ\201Ð»Ð° Ð¾Ñ\202 1 Ð´Ð¾ 100
    for i in range(1, 6):
        file_path = os.path.join('secure_data', f'data_{i}.txt')
        with open(file_path, 'w') as f:
            f.write(str(random.randint(1, 100)))

def calculate_sum_and_create_result():
    total_sum = 0
    found_files = 0

    # Ð§Ð¸Ñ\202Ð°ÐµÐ¼ Ð²Ñ\201Ðµ Ñ\204Ð°Ð¹Ð»Ñ\213 Ð¸Ð• Ð¿Ð°Ð¿Ð¾Ð¸ 'secure_data'
    for i in range(1, 6):
        file_path = os.path.join('secure_data', f'data_{i}.txt')
        if os.path.exists(file_path):
            with open(file_path, 'r') as f:
                try:
                    total_sum += int(f.read().strip())
                    found_files += 1
                except ValueError:
                    print(f"Ð\237Ñ\200Ð¾Ð¿Ñ\203Ñ\201Ð°Ð°ÐµÐ¼ Ð¿Ð¾Ð²Ñ\200ÐµÐ¶Ð´ÐµÐ½Ð½Ñ\213
Ð¹ Ñ\204Ð°Ð¹Ð»: {file_path}")
        else:
            print(f"Ð\237Ñ\200Ð¾Ð¿Ñ\203Ñ\201Ð°Ð°ÐµÐ¼ Ð¾Ñ\202Ñ\201Ñ\203Ñ\202Ñ\201Ñ\202Ð²Ñ
\203Ñ\216Ñ\211Ð¸Ð¹ Ñ\204Ð°Ð¹Ð»: {file_path}")

    # Ð¡Ð¾Ð·Ð´Ð°ÐµÐ¼ Ñ\204Ð°Ð¹Ð» 'result.txt' Ñ\201 Ñ\201Ð¾Ð¾Ñ\202Ð²ÐµÑ\202Ñ\201Ñ\202Ð²Ñ
\203Ñ\216Ñ\211Ð¸Ð¼ Ñ\201Ð¾Ð¾Ð±Ñ\211ÐµÐ½Ð¸ÐµÐ¼
    if found_files == 5:
```

```
        with open('result.txt', 'w') as f:
            if total_sum > 250:
                f.write('HIGH')
            else:
                f.write('LOW')

# Р\222Ñ\213Р•Ñ\213Р²Р°РµР¼ Ñ\204Ñ\203Р½Р°Ñ\206Р¸Р¸
create_folder_and_files()
calculate_sum_and_create_result()-e


=== FILE: ./success/success_2026-10-25_13-45-32.py ===
import datetime

# Р¡Р¾Р•Р´Р°Р½Рµ Ñ\204Р°Р¹Р»Р° 'server.log' Р¸ Р•Р°Р¿Р¸Ñ\201Ñ\214 Р² Р½РµР³Р¾ 10 Ñ\201Ñ
\202Ñ\200Р¾Р° Р»Р¾Р³Р°
with open('server.log', 'w') as log_file:
    for i in range(10):
        log_file.write(f"{datetime.datetime(2026, 10, 25, 14, i)} [INFO] Us\033[2D\033[K
User logged in\n")
        log_file.write(f"{datetime.datetime(2026, 10, 25, 14, i)} [ERROR] D\033[1D\033[K
Database connection failed\n")
        log_file.write(f"{datetime.datetime(2026, 10, 25, 14, i)} [WARNING]\033[9D\033[K
[WARNING] Disk space is low\n")

# Р§Ñ\202РµРµ Ñ\204Р°Р¹Р»Р° Р¸ Ñ\200Р°Ñ\201Р¿Р°Ñ\200Ñ\201Р¸Р²Р³ Ñ\201Ñ\202Ñ\200Р¾Р°
error_count = 0
warning_count = 0

try:
    with open('server.log', 'r') as log_file:
        for line in log_file:
            try:
                timestamp, _, level, message = line.split()
                if level == 'ERROR':
                    error_count += 1
                elif level == 'WARNING':
                    warning_count += 1
            except ValueError:
                continue

except FileNotFoundError:
    print("Error: File not found")

# Р\222Ñ\213Р²Р¾Р´ Ñ\200РµР•Ñ\203Р»Ñ\214Ñ\202Р°Ñ\202Р°
print(f"ERROR: {error_count}, WARNING: {warning_count}")-e


=== FILE: ./success/success_2026-10-25_13-49-18.py ===
import json

class TaskManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, title, priority):
        if priority not in ['low', 'medium', 'high']:
            raise ValueError("Priority must be 'low', 'medium', or 'high'."\033[8D\033[K
'high'.")
        self.tasks.append({'title': title, 'priority': priority})

    def get_tasks_by_priority(self, priority):
        return [task for task in self.tasks if task['priority'] == priority\033[8D\033[K
priority]

    def save_to_json(self, filename):
        with open(filename, 'w') as file:
            json.dump(self.tasks, file)

    def load_from_json(self, filename):
```

```
        try:
            with open(filename, 'r') as file:
                self.tasks = json.load(file)
        except FileNotFoundError:
            print("File not found. Creating a new list of tasks.")
            self.tasks = []

# Example usage
task_manager = TaskManager()
task_manager.add_task("Task 1", "high")
task_manager.add_task("Task 2", "medium")
task_manager.add_task("Task 3", "low")
task_manager.save_to_json("tasks.json")

task_manager.load_from_json("tasks.json")
high_priority_tasks = task_manager.get_tasks_by_priority("high")
for task in high_priority_tasks:
    print(f"Title: {task['title']}, Priority: {task['priority']}")-e


=== FILE: ./success/success_2026-10-25_13-53-01.py ===
class PluginNotFoundError(Exception):
    pass

class Plugin:
    def execute(self, data):
        raise NotImplementedError("This method should be implemented in sub\033[3D\033[K
subclasses")

class UpperCasePlugin(Plugin):
    def execute(self, data):
        return data.upper()

class ReversePlugin(Plugin):
    def execute(self, data):
        return data[::-1]

class PluginManager:
    def __init__(self):
        self.plugins = {}

    def register_plugin(self, name, plugin):
        if name in self.plugins:
            raise ValueError(f"Plugin with name '{name}' already exists")
        self.plugins[name] = plugin

    def run_plugin(self, name, data):
        if name not in self.plugins:
            raise PluginNotFoundError(f"Plugin '{name}' not found")
        return self.plugins[name].execute(data)

# Ð¡Ð¾Ð·Ð´Ð°Ð½Ð¸Ðµ Ñ\215Ð°Ð•ÐµÐ¼Ð¿Ð»Ñ\217Ñ\200Ð° Ð¼ÐµÐ½ÐµÐ´Ð¶ÐµÑ\200Ð°
manager = PluginManager()

# Ð ÐµÐ³Ð¸Ñ\201Ñ\202Ñ\200Ð°Ñ\206Ð¸Ñ\217 Ð¿Ð»Ð°Ð³Ð¸Ð½Ð¾Ð²
manager.register_plugin('upper_case', UpperCasePlugin())
manager.register_plugin('reverse', ReversePlugin())

# Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ð¿Ð»Ð°Ð³Ð¸Ð½Ð¾Ð²
result_upper = manager.run_plugin('upper_case', 'hello')
result_reverse = manager.run_plugin('reverse', 'world')

# Ð\237Ð¾Ð¿Ñ\213Ñ\202Ð°Ð• Ð•Ð°Ð¿Ñ\203Ñ\201Ñ\202Ð¸Ñ\202Ñ\214 Ð½ÐµÑ\201Ñ\203Ñ\211ÐµÑ\201Ñ
\202Ð²Ñ\203Ñ\216Ñ\211Ð¸Ð¹ Ð¿Ð»Ð°Ð³Ð¸Ð½
try:
    result_unknown = manager.run_plugin('unknown', 'test')
except PluginNotFoundError as e:
    print(e)-e
```

```
=== FILE: ./success/success_2026-10-25_13-58-19.py ===
import aiohttp
import asyncio
import logging

logging.basicConfig(level=logging.INFO)

async def fetch(session, url):
    try:
        async with session.get(url, timeout=5) as response:
            logging.info(f"Fetching {url}... {response.status}")
            response.raise_for_status()
            return len(await response.read())
    except (aiohttp.ClientError, asyncio.TimeoutError) as e:
        logging.error(f"Error fetching {url}: {e}")
        return None

async def main(urls):
    semaphore = asyncio.Semaphore(2)
    results = {}

    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        for task in asyncio.as_completed(tasks, limit=2):
            url = task.get_name()
            size = await task
            results[url] = size

    return results

urls = [
    "http://example.com",
    "http://example.org",
    "http://example.net",
    "http://example.edu",
    "http://example.com"
]

loop = asyncio.get_event_loop()
result = loop.run_until_complete(main(urls))
print(result)-e


=== FILE: ./success/success_2026-10-25_14-11-43.py ===
Ð\230Ð•Ð²Ð¸Ð½Ñ\202Ðµ, Ð½Ð¾ Ñ\217 Ð½Ðµ Ð¼Ð¾Ð³Ñ\203 Ð¿Ð¾Ð¼Ð¾Ñ\207Ñ\214 Ñ\201 Ñ\215Ñ\202Ð¸
Ð¼ Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð¾Ð¼.-e


=== FILE: ./success/success_2026-10-25_14-23-30.py ===
import threading
import time
from collections import OrderedDict

class TTLCache:
    def __init__(self, capacity: int, default_ttl: float, cleanup_interval:\033[17D\033[K
cleanup_interval: float):
        self.capacity = capacity
        self.default_ttl = default_ttl
        self.cleanup_interval = cleanup_interval
        self.cache = OrderedDict()
        self.lock = threading.Lock()
        self.hits = 0
        self.misses = 0
        self.evictions = 0
        self.expirations = 0
        self.on_evict = None

    def get(self, key):
        with self.lock:
            if key in self.cache:
```

```
                self.hits += 1
                self._update_ttl(key)
                return self.cache[key]
            else:
                self.misses += 1
                return None

    def put(self, key, value, ttl=None):
        with self.lock:
            if key in self.cache:
                self.evictions += 1
                self.cache.pop(key)
            self._update_ttl(key, ttl or self.default_ttl)
            self.cache[key] = value
            if len(self.cache) > self.capacity:
                self._evict_lru()

    def _update_ttl(self, key, ttl=None):
        if ttl is not None:
            self.cache.move_to_end(key, last=False)
        else:
            self.cache.move_to_end(key)

    def _evict_lru(self):
        with self.lock:
            while len(self.cache) > self.capacity:
                key, _ = self.cache.popitem(last=False)
                self.expirations += 1
                if self.on_evict:
                    self.on_evict(key, self.cache[key], "lru")

    def start_cleanup(self):
        while True:
            with self.lock:
                if time.time() - self.last_cleanup > self.cleanup_interval:\033[22D\033[K
self.cleanup_interval:
                    self._evict_expired()
            time.sleep(self.cleanup_interval)

    def _evict_expired(self):
        with self.lock:
            for key in list(self.cache.keys()):
                if time.time() - self.cache[key] > self.default_ttl:
                    self.evictions += 1
                    if self.on_evict:
                        self.on_evict(key, self.cache[key], "ttl")
                    self.cache.pop(key)

    def get_stats(self):
        with self.lock:
            return {
                "hits": self.hits,
                "misses": self.misses,
                "evictions": self.evictions,
                "expirations": self.expirations,
                "size": len(self.cache),
                "hit_ratio": self.hits / (self.hits + self.misses) if self.\033[5D\033[K
self.hits + self.misses > 0 else 0
            }

    def set_on_evict(self, callback):
        self.on_evict = callback

# Đ¢ĐµÑ\201Ñ\202
if __name__ == "__main__":
    cache = TTLCache(capacity=100, default_ttl=0.5, cleanup_interval=0.1)
    cache.set_on_evict(lambda key, value, reason: print(f"Evicted: {key} du\033[2D\033[K
due to {reason}"))

    threads = []
```

```
    for _ in range(8):
        thread = threading.Thread(target=lambda: [
            cache.get(i % 50),
            cache.put(i % 50, i, 0.05 + 0.15 * (i % 50))
            for i in range(500)
        ])
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    stats = cache.get_stats()
    assert stats["hits"] + stats["misses"] == 4000
    assert stats["size"] <= 100
    assert not cache.on_evict_called  # Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Đ°Đ° Ñ\202Đ¾Đ³Đ¾, Ñ\207Ñ\202
Đ¾ on_evict Đ½Đµ Đ²Ñ\213Đ•Ñ\213\033[4D\033[K
Đ²Ñ\213Đ•Ñ\213Đ²Đ°Đ»Ñ\201Ñ\217 Đ¿Đ¾Đ´ Đ»Đ¾Đ°Đ¾Đ¼

    print(stats)-e


=== FILE: ./success/clean_cache.py ===
import threading
import time
from collections import OrderedDict


class TTLCache:
    def __init__(self, capacity, default_ttl, cleanup_interval):
        self.capacity = capacity
        self.default_ttl = default_ttl
        self.cleanup_interval = cleanup_interval
        self.cache = OrderedDict()
        self.lock = threading.RLock()
        self.hits = 0
        self.misses = 0
        self.evictions = 0
        self.expirations = 0
        self.on_evict = None
        self._stop_event = threading.Event()
        self._cleaner_thread = None

    def get(self, key):
        victims = []
        with self.lock:
            item = self.cache.get(key)
            if item is None:
                self.misses += 1
                result = None
            else:
                value, expires_at = item
                if expires_at is not None and time.monotonic() >= expires_at:
                    self.cache.pop(key)
                    self.expirations += 1
                    self.misses += 1
                    victims.append((key, value, "ttl"))
                    result = None
                else:
                    self.hits += 1
                    self.cache.move_to_end(key)
                    result = value
        self._fire_callbacks(victims)
        return result

    def put(self, key, value, ttl=None):
        victims = []
        ttl = self.default_ttl if ttl is None else ttl
        expires_at = (time.monotonic() + ttl) if ttl is not None else None
        with self.lock:
```

```
            if key in self.cache:
                self.cache.pop(key)
            self.cache[key] = (value, expires_at)
            while len(self.cache) > self.capacity:
                old_key, (old_value, _) = self.cache.popitem(last=False)
                self.evictions += 1
                victims.append((old_key, old_value, "lru"))
        self._fire_callbacks(victims)

    def get_stats(self):
        with self.lock:
            total = self.hits + self.misses
            return {
                "hits": self.hits,
                "misses": self.misses,
                "evictions": self.evictions,
                "expirations": self.expirations,
                "size": len(self.cache),
                "hit_ratio": self.hits / total if total else 0.0,
            }

    def set_on_evict(self, callback):
        with self.lock:
            self.on_evict = callback

    def start_cleanup(self):
        if self._cleaner_thread is not None:
            return
        self._cleaner_thread = threading.Thread(
            target=self._cleanup_loop, daemon=True
        )
        self._cleaner_thread.start()

    def stop_cleanup(self):
        self._stop_event.set()
        if self._cleaner_thread is not None:
            self._cleaner_thread.join(timeout=2.0)
            self._cleaner_thread = None

    def _cleanup_loop(self):
        while not self._stop_event.wait(self.cleanup_interval):
            self._cleanup_once()

    def _cleanup_once(self):
        now = time.monotonic()
        expired = []
        with self.lock:
            for key, (value, expires_at) in list(self.cache.items()):
                if expires_at is not None and now >= expires_at:
                    self.cache.pop(key)
                    self.expirations += 1
                    expired.append((key, value, "ttl"))
        self._fire_callbacks(expired)

    def _fire_callbacks(self, victims):
        cb = self.on_evict
        if cb is None:
            return
        for key, value, reason in victims:
            try:
                cb(key, value, reason)
            except Exception:
                pass


def main():
    import random

    cache = TTLCache(capacity=100, default_ttl=0.5, cleanup_interval=0.1)
```

```
    violations = []
    vlock = threading.Lock()

    def on_evict(key, value, reason):
        if cache.lock.locked():
            with vlock:
                violations.append((key, reason))

    cache.set_on_evict(on_evict)
    cache.start_cleanup()

    THREADS = 8
    OPS = 500
    KEYS = 50
    total_gets_counter = [0]
    gets_lock = threading.Lock()

    def worker(seed):
        rnd = random.Random(seed)
        local_gets = 0
        for _ in range(OPS):
            key = rnd.randrange(KEYS)
            if rnd.random() < 0.5:
                cache.get(key)
                local_gets += 1
            else:
                ttl = rnd.choice([0.05, 0.1, 0.2, 0.5, None])
                cache.put(key, key, ttl=ttl)
        with gets_lock:
            total_gets_counter[0] += local_gets

    threads = [threading.Thread(target=worker, args=(i,)) for i in range(THREADS)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

    cache.stop_cleanup()
    stats = cache.get_stats()
    print("Stats:", stats)

    total_gets = total_gets_counter[0]
    assert stats["hits"] + stats["misses"] == total_gets, (
        "hits+misses=%d != total_gets=%d" % (
            stats["hits"] + stats["misses"], total_gets
        )
    )
    assert stats["size"] <= cache.capacity, (
        "size=%d > capacity=%d" % (stats["size"], cache.capacity)
    )
    assert not violations, "callback under lock: %r" % (violations[:5],)
    print("OK: all invariants satisfied")


if __name__ == "__main__":
    main()
-e


=== FILE: ./trash/trash_final.py ===
-e


=== FILE: ./trash/appeal_1.py ===
import datetime
import os
import subprocess
import sys
import re
```

```
def smart_alarm(time_str, reminder_text):
    # Check if the time format is correct
    if not re.match(r'^([01]?[0-9]|2[0-3]):([0-5][0-9])$', time_str):
        print("Ð\235ÐµÐ²ÐµÑ\200Ð½Ð¾Ðµ Ð²Ñ\200ÐµÐ¼Ñ\217")
        return

    # Parse the time string
    try:
        alarm_time = datetime.datetime.strptime(time_str, "%H:%M")
    except ValueError:
        print("Ð\235ÐµÐ²ÐµÑ\200Ð½Ð¾Ðµ Ð²Ñ\200ÐµÐ¼Ñ\217")
        return

    # Get the current time
    current_time = datetime.datetime.now()

    # Calculate the time difference
    time_difference = alarm_time - current_time

    # If the alarm time is already passed, adjust to the next day
    if time_difference.total_seconds() < 0:
        alarm_time += datetime.timedelta(days=1)

    # Wait until the alarm time
    time.sleep(time_difference.total_seconds())

    # Use termux-tts-speak to speak the reminder
    subprocess.run(["termux-tts-speak", reminder_text or "Ð\237Ð¾Ñ\200Ð° Ð²Ñ\201Ñ\202Ð°Ð²Ð°Ñ\202Ñ\214!"])\033[12D\033[K
Ð²Ñ\201Ñ\202Ð°Ð²Ð°Ñ\202Ñ\214!"])

# Example usage
smart_alarm("07:30", "Ð\237Ð¾Ñ\200Ð° Ð²Ñ\201Ñ\202Ð°Ð²Ð°Ñ\202Ñ\214!")-e


=== FILE: ./trash/trash_1.py ===
import datetime

with open('report.txt', 'w') as file:
    file.write(datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S'))-e


=== FILE: ./trash/trash_2.py ===
import os
import subprocess
import time

# Ð¡Ð¾Ð·Ð´Ð°Ñ‚ÑŒ Ñ\204Ð°Ð¹Ð»Ð° Ð¸ Ð•Ð°Ð¿Ð¸Ñ\201Ñ\214 Ñ\202ÐµÐ°Ñ\203Ñ\211ÐµÐ¹ Ð´Ð°Ñ\202Ñ\213 Ð¸ Ð²Ñ\200ÐµÐ¼ÐµÐ½Ð¸
with open('report.txt', 'w') as f:
    f.write(f"{time.strftime('%Y-%m-%d %H:%M:%S')}")

print("Ð¤Ð°Ð¹Ð» report.txt Ñ\201Ð¾Ð·Ð´Ð°Ð½ Ð¸ Ð•Ð°Ð¿Ð¾Ð»Ð½ÐµÐ½.")-e


=== FILE: ./trash/trash_3.py ===
import os
import time

# Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ñ\217ÐµÐ¼, Ñ\201Ñ\203Ñ\211ÐµÑ\201Ñ\202Ð²Ñ\203ÐµÑ\202 Ð»Ð¸ Ñ\204Ð°Ð¹Ð» report.txt
if not os.path.exists('report.txt'):
    # Ð\225Ñ\201Ð»Ð¸ Ñ\204Ð°Ð¹Ð»Ð° Ð½ÐµÑ Ñ\201Ñ\203Ñ\211ÐµÑ\201Ñ\202Ð²Ñ\203ÐµÑ\202, Ñ\201Ð¾Ð·Ð´Ð°ÐµÐ¼ ÐµÐ³Ð¾
    with open('report.txt', 'w') as file:
        # Ð\227Ð°Ð¿Ð¸Ñ\201Ñ\213Ð²Ð°ÐµÐ¼ Ñ\202ÐµÐ°Ñ\203Ñ\211Ñ\203Ñ\216 Ð´Ð°Ñ\202Ñ\203 Ð¸ Ð²Ñ\200ÐµÐ¼Ñ\217
        file.write(f"{time.strftime('%Y-%m-%d %H:%M:%S')}\n")-e
```

```
=== FILE: ./wip/wip_1.py ===
-e


=== FILE: ./wip/wip_2.py ===
-e


=== FILE: ./wip/wip_3.py ===
-e


=== FILE: ./appeal/appeal_1.py ===
import random
import sys

try:
    random_number = random.randint(10, 50)
    print(random_number)
except Exception as e:
    print(f"An error occurred: {e}", file=sys.stderr)-e


=== FILE: ./appeal/appeal_2.py ===
import datetime

def write_report():
    try:
        with open("report.txt", "w") as file:
            file.write(datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"\033[9D\033[K
%H:%M:%S"))
    except IOError as e:
        print(f"Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð•Ð°Ð¿Ð¸Ñ\201Ð¸ Ð² Ñ\204Ð°Ð¹Ð»: {e}")
    except Exception as e:
        print(f"Ð\237Ñ\200Ð¾Ð¸Ð·Ð¾Ñ\210Ð»Ð° Ð½ÐµÐ¸Ð•Ð²ÐµÑ\201Ñ\202Ð½Ð°Ñ\217 Ð¾Ñ\210Ð¸Ð±Ð°
Ð°: {e}")

write_report()-e


=== FILE: ./appeal/appeal_3.py ===
import os
import datetime
from time import time

def func(path):
    try:
        with open(path, 'w+') as f:
            f.write(str(datetime.datetime.now()))
    except Exception as e:
        print(f"Error writing to file: {e}")
    print(str(datetime.datetime.now()))
    return

func(os.getcwd()+'/report.txt')-e


=== FILE: ./critic_logic.py ===
import os
import subprocess

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft_analyzed.py")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
REPORT_FILE = os.path.join(FACTORY_DIR, "logic_errors.txt")
MODEL = "llama3.2:1b"

def main():
    if not os.path.exists(DRAFT_FILE) or not os.path.exists(TASK_FILE):
        return
```

```
    with open(DRAFT_FILE, "r", encoding="utf-8") as f:
        code = f.read().strip()
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()

    prompt = (
        f"Đ\227Đ°Đ´Đ°Ñ\207Đ°: {task}\n"
        "Đ¢Ñ\213 â\200\224 Đ\233Đ¾Đ³Đ¸Đ°. Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Ñ\214, Ñ\200ĐµÑ\210Đ°ĐµÑ
\202 Đ»Đ¸ Ñ\215Ñ\202Đ¾Ñ\202 Đ°Đ¾Đ´ Đ•Đ°Đ´Đ°Ñ\207Ñ\203 Đ¸ Đ½ĐµÑ\202 Đ»Đ¸ Đ»Đ¾Đ³Đ¸Ñ\207ĐµÑ
\201Đ°Đ¸Ñ\205 Đ¾Ñ\210Đ¸Đ±Đ¾Đ°. "
        "Đ\225Ñ\201Đ»Đ¸ Đ²Ñ\201Ñ\221 Ñ\205Đ¾Ñ\200Đ¾Ñ\210Đ¾ â\200\224 Đ½Đ°Đ¿Đ¸Ñ\210Đ¸ 'Đ£Đ
¡Đ\237Đ\225Đ¥'. Đ\225Ñ\201Đ»Đ¸ ĐµÑ\201Ñ\202Ñ\214 Đ¾Ñ\210Đ¸Đ±Đ°Đ¸ â\200\224 'Đ\236Đ¨Đ\230Đ
\221Đ\232Đ\220: [Đ¿Ñ\200Đ¸Ñ\207Đ¸Đ½Đ°]'. "
        f"Đ\232Đ¾Đ´:\n{code}"
    )

    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt, "--num-predict", "200"],
            capture_output=True, text=True, timeout=600
        )
        with open(REPORT_FILE, "w", encoding="utf-8") as f:
            f.write(result.stdout.strip())
    except Exception as e:
        print(f"[CRITIC_LOGIC] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: {e}")

if __name__ == "__main__":
    main()
-e


=== FILE: ./critic_syntax.py ===
import os
import subprocess

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft_analyzed.py")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
REPORT_FILE = os.path.join(FACTORY_DIR, "syntax_errors.txt")
MODEL = "qwen2.5:1.5b"

def main():
    if not os.path.exists(DRAFT_FILE) or not os.path.exists(TASK_FILE):
        return
    with open(DRAFT_FILE, "r", encoding="utf-8") as f:
        code = f.read().strip()
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()

    prompt = (
        f"Đ\227Đ°Đ´Đ°Ñ\207Đ°: {task}\n"
        "Đ¢Ñ\213 â\200\224 Đ¡Đ¸Đ½Ñ\202Đ°Đ°Ñ\201Đ¸Ñ\207ĐµÑ\201Đ°Đ¸Đ¹ Đ\234Đ¾Đ½Ñ\201Ñ\202Ñ
\200. Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Ñ\214 Đ°Đ¾Đ´ Đ½Đ° Ñ\201Đ¸Đ½Ñ\202Đ°Đ°Ñ\201Đ¸Ñ\207ĐµÑ\201Đ°Đ¸Đµ
Đ¾Ñ\210Đ¸Đ±Đ°Đ¸. "
        "Đ\225Ñ\201Đ»Đ¸ Đ²Ñ\201Ñ\221 Ñ\205Đ¾Ñ\200Đ¾Ñ\210Đ¾ â\200\224 Đ½Đ°Đ¿Đ¸Ñ\210Đ¸ 'Đ£Đ
¡Đ\237Đ\225Đ¥'. Đ\225Ñ\201Đ»Đ¸ ĐµÑ\201Ñ\202Ñ\214 Đ¾Ñ\210Đ¸Đ±Đ°Đ¸ â\200\224 'Đ\236Đ¨Đ\230Đ
\221Đ\232Đ\220: [Đ¿Ñ\200Đ¸Ñ\207Đ¸Đ½Đ°]'. "
        f"Đ\232Đ¾Đ´:\n{code}"
    )

    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt, "--num-predict", "200"],
            capture_output=True, text=True, timeout=600
        )
        with open(REPORT_FILE, "w", encoding="utf-8") as f:
            f.write(result.stdout.strip())
    except Exception as e:
        print(f"[CRITIC_SYNTAX] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: {e}")
```

```
if __name__ == "__main__":
    main()
-e


=== FILE: ./critic_coder.py ===
import os
import subprocess

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft_analyzed.py")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
REPORT_FILE = os.path.join(FACTORY_DIR, "coder_errors.txt")
MODEL = "stable-code:3b"

def main():
    if not os.path.exists(DRAFT_FILE) or not os.path.exists(TASK_FILE):
        return
    with open(DRAFT_FILE, "r", encoding="utf-8") as f:
        code = f.read().strip()
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()

    prompt = (
        f"Đ\227Đ°Đ´Đ°Ñ\207Đ°: {task}\n"
        "Đ¢Ñ\213 â\200\224 Đ¾Đ¿Ñ\213Ñ\202Đ½Ñ\213Đ¹ Đ¿Ñ\200Đ¾Đ³Ñ\200Đ°Đ¼Đ¼Đ¸Ñ\201Ñ\202. Đ
\237Ñ\200Đ¾Đ²ĐµÑ\200Ñ\214 Đ¾Đ±Ñ\211ĐµĐµ Đ°Đ°Ñ\207ĐµÑ\201Ñ\202Đ²Đ¾ Đ°Đ¾Đ´Đ° Đ¸ Ñ\202Đ¾, Ñ
\200ĐµÑ\210Đ°ĐµÑ\202 Đ»Đ¸ Đ¾Đ½ Đ•Đ°Đ´Đ°Ñ\207Ñ\203. "
        "Đ\225Ñ\201Đ»Đ¸ Đ²Ñ\201Ñ\221 Ñ\205Đ¾Ñ\200Đ¾Ñ\210Đ¾ â\200\224 Đ½Đ°Đ¿Đ¸Ñ\210Đ¸ 'Đ£Đ
¡Đ\237Đ\225Đ¥'. Đ\225Ñ\201Đ»Đ¸ ĐµÑ\201Ñ\202Ñ\214 Đ•Đ°Đ¼ĐµÑ\207Đ°Đ½Đ¸Ñ\217 â\200\224 'Đ
\236Đ¨Đ\230Đ\221Đ\232Đ\220: [Đ•Đ°Đ¼ĐµÑ\207Đ°Đ½Đ¸Đµ]'. "
        f"Đ\232Đ¾Đ´:\n{code}"
    )

    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt, "--num-predict", "200"],
            capture_output=True, text=True, timeout=600
        )
        with open(REPORT_FILE, "w", encoding="utf-8") as f:
            f.write(result.stdout.strip())
    except Exception as e:
        print(f"[CRITIC_CODER] Đ\236Ñ\210Đ¸Đ±Đ°Đ°: {e}")

if __name__ == "__main__":
    main()
-e


=== FILE: ./judge.py ===
import os
import subprocess
import re

# --- Đ\235Đ\220Đ¡Đ¢Đ Đ\236Đ\231Đ\232Đ\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
DRAFT_FILE = os.path.join(FACTORY_DIR, "draft_analyzed.py")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
DRAFT_FINAL = os.path.join(FACTORY_DIR, "draft_final.py")
LOGIC_ERRORS = os.path.join(FACTORY_DIR, "logic_errors.txt")
SYNTAX_ERRORS = os.path.join(FACTORY_DIR, "syntax_errors.txt")
CODER_ERRORS = os.path.join(FACTORY_DIR, "coder_errors.txt")
MODEL = "qwen2.5-coder:3b"

def read_file(path):
    if not os.path.exists(path):
        return ""
    try:
        with open(path, "r", encoding="utf-8") as f:
            return f.read().strip()
```

```
    except Exception:
        return ""

def clean_code(text):
    match = re.search(r'```(?:python)?\s*(.*?)```', text, re.DOTALL)
    if match:
        return match.group(1).strip()
    return text.strip()

def ask_judge(prompt, step_name):
    """Р\236Ñ\202Р¿Ñ\200Р°Р²Р»Ñ\217РµÑ\202 Р•Р°Р¿Ñ\200Р¾Ñ\201 Р¡Ñ\203Р´Ñ\214Рµ Р¸ Р²Р¾Р•Р
²Ñ\200Р°Ñ\211Р°РµÑ\202 Р¾Ñ\207Р¸Ñ\211РµÐ½Ð½Ñ\213Р¹ Р°Р¾Р´."""
    print(f"[JUDGE] Р¨Р°Р³: {step_name}...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt, "--num-predict", "500"],
            capture_output=True, text=True, timeout=600
        )
        if result.returncode != 0:
            print(f"[JUDGE] Р\236Ñ\210Р¸Р±Р°Р° Ollama Р½Р° Ñ\210Р°Р³Рµ {step_name}.")
            return None
        clean = clean_code(result.stdout)
        return clean if clean else None
    except Exception as e:
        print(f"[JUDGE] Р\230Ñ\201Р°Р»Ñ\216Ñ\207РµÐ½Ð¸Рµ Р½Р° Ñ\210Р°Р³Рµ {step_name}: {e
}")
        return None

def main():
    # 1. Р§Р¸Ñ\202Р°РµÐ¼ Р¸Ñ\201Ñ\205Р¾Р´Ð½Ñ\213Рµ Р´Р°Ð½Ð½Ñ\213Рµ
    if not os.path.exists(DRAFT_FILE) or not os.path.exists(TASK_FILE):
        print("[JUDGE] Р\236Ñ\210Р¸Р±Р°Р°: Р½ÐµÑ\202 draft_analyzed.py Р¸Р»Р¸ task.txt")
        return

    with open(DRAFT_FILE, "r", encoding="utf-8") as f:
        current_code = f.read().strip()
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()

    if not current_code:
        print("[JUDGE] Р\236Ñ\210Р¸Р±Р°Р°: draft_analyzed.py Р¿Ñ\203Ñ\201Ñ\202.")
        return

    # 2. Р§Р¸Ñ\202Р°РµÐ¼ Р¾Ñ\210Р¸Р±Р°Р¸ Р¾Ñ\202 Р°Ñ\200Р¸Ñ\202Р¸Р°Р¾Р²
    logic_err = read_file(LOGIC_ERRORS)
    syntax_err = read_file(SYNTAX_ERRORS)
    coder_err = read_file(CODER_ERRORS)

    # 3. Р\237Р¾Ñ\210Р°Р³Р¾Р²Р¾Рµ Р¸Ñ\201Р¿Ñ\200Р°Р²Р»ÐµÐ½Ð¸Рµ (Р¡Р°Р»Р°Ð´Ñ\213)

    # Р¡Р°Р»Р°Ð´ 1: Р\233Р¾Р³Р¸Р°Р°
    if logic_err and "Р£Р¡Р\237Р\225Р¥" not in logic_err.upper():
        prompt = (
            f"Р\227Р°Ð´Р°Ñ\207Р°: {task}\n"
            f"Р\232Р¾Ð´:\n{current_code}\n"
            f"Р\232Ñ\200Р¸Ñ\202Р¸Р°-Р\233Р¾Р³Р¸Р° Р½Р°Ñ\210ÐµÐ» Р¾Ñ\210Р¸Р±Р°Ñ\203: {logi
c_err}\n"
            "Р\230Ñ\201Р¿Ñ\200Р°Р²Ñ\214 Р¢Р\236Р\233Р¬Р\232Р\236 Ñ\215Ñ\202Ñ\203 Ð»Ð¾Ð³Р¸
Ñ\207ÐµÑ\201Р°Ñ\203Ñ\216 Р¾Ñ\210Р¸Р±Р°Ñ\203. Р\235Р°Р¿Р¸Ñ\210Р¸ Ñ\202Р¾Ð»Ñ\214Р°Р¾ Р°Р¾Ð´
."
        )
        new_code = ask_judge(prompt, "Р\230Ñ\201Р¿Ñ\200Р°Р²Р»ÐµÐ½Ð¸Рµ Р\233Р¾Р³Р¸Р°Р¸")
        if new_code: current_code = new_code

    # Р¡Р°Р»Р°Ð´ 2: Р¡Р¸Ð½Ñ\202Р°Р°Ñ\201Р¸Ñ\201
    if syntax_err and "Р£Р¡Р\237Р\225Р¥" not in syntax_err.upper():
        prompt = (
            f"Р\227Р°Ð´Р°Ñ\207Р°: {task}\n"
            f"Р\232Р¾Ð´:\n{current_code}\n"
            f"Р\232Ñ\200Р¸Ñ\202Р¸Р°-Р¡Р¸Ð½Ñ\202Р°Р°Ñ\201Р¸Ñ\201 Р½Р°Ñ\210ÐµÐ» Р¾Ñ\210Р¸Р±
```

```
Đ°Ñ\203: {syntax_err}\n"
            "Đ\230Ñ\201Đ¿Ñ\200Đ°Đ²Ñ\214 Đ¢Đ\236Đ\233Đ¬Đ\232Đ\236 Ñ\215Ñ\202Ñ\203 Ñ\201Đ¸Đ
½Ñ\202Đ°Đ°Ñ\201Đ¸Ñ\207ĐµÑ\201Đ°Ñ\203Ñ\216 Đ¾Ñ\210Đ¸Đ±Đ°Ñ\203. Đ\235Đ°Đ¿Đ¸Ñ\210Đ¸ Ñ\202Đ¾Đ
»Ñ\214Đ°Đ¾ Đ°Đ¾Đ´."
        )
        new_code = ask_judge(prompt, "Đ\230Ñ\201Đ¿Ñ\200Đ°Đ²Đ»ĐµĐ½Đ¸Đµ Đ¡Đ¸Đ½Ñ\202Đ°Đ°Ñ
\201Đ¸Ñ\201Đ°")
        if new_code: current_code = new_code

    # Đ¡Đ°Đ»Đ°Đ´ 3: Đ\236Đ±Ñ\211ĐµĐµ Đ°Đ°Ñ\207ĐµÑ\201Ñ\202Đ²Đ¾
    if coder_err and "Đ£Đ¡Đ\237Đ\225Đ¥" not in coder_err.upper():
        prompt = (
            f"Đ\227Đ°Đ´Đ°Ñ\207Đ°: {task}\n"
            f"Đ\232Đ¾Đ´:\n{current_code}\n"
            f"Đ\232Ñ\200Đ¸Ñ\202Đ¸Đ°-Đ\232Đ¾Đ´ĐµÑ\200 Đ½Đ°Ñ\210ĐµĐ» Đ•Đ°Đ¼ĐµÑ\207Đ°Đ½Đ¸Đµ:
 {coder_err}\n"
            "Đ\230Ñ\201Đ¿Ñ\200Đ°Đ²Ñ\214 Đ¢Đ\236Đ\233Đ¬Đ\232Đ\236 Ñ\215Ñ\202Đ¾ Đ•Đ°Đ¼ĐµÑ
\207Đ°Đ½Đ¸Đµ. Đ\235Đ°Đ¿Đ¸Ñ\210Đ¸ Ñ\202Đ¾Đ»Ñ\214Đ°Đ¾ Đ°Đ¾Đ´."
        )
        new_code = ask_judge(prompt, "Đ\230Ñ\201Đ¿Ñ\200Đ°Đ²Đ»ĐµĐ½Đ¸Đµ Đ\232Đ¾Đ´ĐµÑ\200Đ°"
)
        if new_code: current_code = new_code

    # Đ¡Đ°Đ»Đ°Đ´ 4: Đ¤Đ¸Đ½Đ°Đ»Ñ\214Đ½Đ°Ñ\217 Ñ\201Đ²ĐµÑ\200Đ°Đ° Ñ\201 Đ•Đ°Đ´Đ°Ñ\207ĐµĐ¹
    prompt = (
        f"Đ\227Đ°Đ´Đ°Ñ\207Đ°: {task}\n"
        f"Đ¢ĐµĐ°Ñ\203Ñ\211Đ¸Đ¹ Đ°Đ¾Đ´:\n{current_code}\n"
        "Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Ñ\214, Ñ\200ĐµÑ\210Đ°ĐµÑ\202 Đ»Đ¸ Ñ\215Ñ\202Đ¾Ñ\202 Đ°Đ¾Đ´
Đ•Đ°Đ´Đ°Ñ\207Ñ\203. Đ\225Ñ\201Đ»Đ¸ Đ¾Đ½ Đ¸Đ´ĐµĐ°Đ»ĐµĐ½ â\200\224 Đ²ĐµÑ\200Đ½Đ¸ ĐµĐ³Đ¾ Đ±Đ
µĐ• Đ¸Đ•Đ¼ĐµĐ½ĐµĐ½Đ¸Đ¹. "
        "Đ\225Ñ\201Đ»Đ¸ ĐµÑ\201Ñ\202Ñ\214 Đ¿Ñ\200Đ¾Đ±Đ»ĐµĐ¼Ñ\213 â\200\224 Đ¸Ñ\201Đ¿Ñ\200
Đ°Đ²Ñ\214 Đ¸Ñ\205. Đ\235Đ°Đ¿Đ¸Ñ\210Đ¸ Ñ\202Đ¾Đ»Ñ\214Đ°Đ¾ Ñ\204Đ¸Đ½Đ°Đ»Ñ\214Đ½Ñ\213Đ¹ Đ°Đ¾
Đ´."
    )
    final_code = ask_judge(prompt, "Đ¤Đ¸Đ½Đ°Đ»Ñ\214Đ½Đ°Ñ\217 Ñ\201Đ²ĐµÑ\200Đ°Đ°")

    if final_code:
        current_code = final_code

    # 4. Đ¡Đ¾Ñ\205Ñ\200Đ°Đ½Ñ\217ĐµĐ¼ Ñ\200ĐµĐ•Ñ\203Đ»Ñ\214Ñ\202Đ°Ñ\202
    try:
        with open(DRAFT_FINAL, "w", encoding="utf-8") as f:
            f.write(current_code)
        print(f"[JUDGE] Đ¤Đ¸Đ½Đ°Đ»Ñ\214Đ½Ñ\213Đ¹ Đ°Đ¾Đ´ Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½ Đ² draf
t_final.py ({len(current_code)} Ñ\201Đ¸Đ²Đ¾Đ»Đ¾Đ²).")
    except Exception as e:
        print(f"[JUDGE] Đ\236Ñ\210Đ¸Đ±Đ°Đ° Đ¿Ñ\200Đ¸ Ñ\201Đ¾Ñ\205Ñ\200Đ°Đ½ĐµĐ½Đ¸Đ¸: {e}")

if __name__ == "__main__":
    main()
-e


=== FILE: ./idealist.py ===
import os
import subprocess
import sys

# --- Đ\235Đ\220Đ¡Đ¢Đ Đ\236Đ\231Đ\232Đ\230 ---
FACTORY_DIR = os.path.expanduser("~/takcas_factory")
SUCCESS_DIR = os.path.join(FACTORY_DIR, "success")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
DIRECTOR_SCRIPT = os.path.join(FACTORY_DIR, "director.py")
MODEL = "qwen2.5:1.5b"

def get_success_tasks():
    """Đ§Đ¸Ñ\202Đ°ĐµÑ\202 Ñ\201Đ¿Đ¸Ñ\201Đ¾Đ° Ñ\204Đ°Đ¹Đ»Đ¾Đ² Đ² success/ (Ñ\207Ñ\202Đ¾Đ±Ñ
\213 Đ½Đµ Đ¿Đ¾Đ²Ñ\202Đ¾Ñ\200Ñ\217Ñ\202Ñ\214Ñ\201Ñ\217)."""
    if not os.path.exists(SUCCESS_DIR):
        return []
```

```
    try:
        files = [f for f in os.listdir(SUCCESS_DIR) if f.endswith(".py")]
        return files
    except Exception as e:
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ñ\207Ñ\202ÐµÐ½Ð¸Ð¸ success/: {e}"
)
        return []

def generate_idea(done_tasks):
    """Ð\223ÐµÐ½ÐµÑ\200Ð¸Ñ\200Ñ\203ÐµÑ\202 Ð¸Ð´ÐµÑ\216 Ñ\207ÐµÑ\200ÐµÐ• Ollama. Ð\222Ð¾Ð•
Ð²Ñ\200Ð°Ñ\211Ð°ÐµÑ\202 Ñ\202ÐµÐ°Ñ\201Ñ\202 Ð•Ð°Ð´Ð°Ñ\207Ð¸ Ð¸Ð»Ð¸ None."""
    done_text = "\n".join(done_tasks) if done_tasks else "(Ð¿Ð¾Ð° Ð½Ð¸Ñ\207ÐµÐ³Ð¾ Ð½ÐµÑ
\202)"

    prompt = (
        "Ð¢Ñ\213 â\200\224 Ð³ÐµÐ½ÐµÑ\200Ð°Ñ\202Ð¾Ñ\200 Ð¸Ð´ÐµÐ¹ Ð´Ð»Ñ\217 Python-Ñ\201Ð°Ñ
\200Ð¸Ð¿Ñ\202Ð¾Ð². "
        "Ð\237Ñ\200Ð¸Ð´Ñ\203Ð¼Ð°Ð¹ Ð\236Ð\224Ð\235Ð£ Ð\236Ð§Ð\225Ð\235Ð¬ Ð\237Ð Ð\236Ð¡Ð¢
Ð£Ð® Ð•Ð°Ð´Ð°Ñ\207Ñ\203 Ð´Ð»Ñ\217 Python-Ñ\201Ð°Ñ\200Ð¸Ð¿Ñ\202Ð°, Ð°Ð¾Ñ\202Ð¾Ñ\200Ñ\203Ñ
\216 Ð¼Ð¾Ð¶Ð½Ð¾ Ñ\200ÐµÑ\210Ð¸Ñ\202Ñ\214 Ð•Ð° 1 Ð¼Ð¸Ð½Ñ\203Ñ\202Ñ\203. "
        "Ð\230Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ñ\203Ð¹ Ñ\202Ð¾Ð»Ñ\214Ð°Ð¾ Ñ\201Ñ\202Ð°Ð½Ð´Ð°Ñ\200Ñ\202Ð½
Ñ\213Ðµ Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð¸ (os, time, datetime, random, math). "
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ñ\205Ð¾Ñ\200Ð¾Ñ\210ÐµÐ¹ Ð•Ð°Ð´Ð°Ñ\207Ð¸: 'Ð\235Ð°Ð¿Ð¸Ñ\210
Ð¸ Ñ\201Ð°Ñ\200Ð¸Ð¿Ñ\202, Ð°Ð¾Ñ\202Ð¾Ñ\200Ñ\213Ð¹ Ñ\201Ð¾Ð•Ð´Ð°ÐµÑ\202 Ñ\204Ð°Ð¹Ð» report
.txt Ð¸ Ð•Ð°Ð¿Ð¸Ñ\201Ñ\213Ð²Ð°ÐµÑ\202 Ñ\202Ñ\203Ð´Ð° Ñ\202ÐµÐ°Ñ\203Ñ\211Ñ\203Ñ\216 Ð´Ð°Ñ
\202Ñ\203 Ð¸ Ð²Ñ\200ÐµÐ¼Ñ\217'. "
        "Ð\237Ñ\200Ð¸Ð¼ÐµÑ\200 Ñ\205Ð¾Ñ\200Ð¾Ñ\210ÐµÐ¹ Ð•Ð°Ð´Ð°Ñ\207Ð¸: 'Ð\235Ð°Ð¿Ð¸Ñ\210
Ð¸ Ñ\201Ð°Ñ\200Ð¸Ð¿Ñ\202, Ð°Ð¾Ñ\202Ð¾Ñ\200Ñ\213Ð¹ Ð³ÐµÐ½ÐµÑ\200Ð¸Ñ\200Ñ\203ÐµÑ\202 5 Ñ
\201Ð»Ñ\203Ñ\207Ð°Ð¹Ñ\213Ñ\205 Ñ\207Ð¸Ñ\201ÐµÐ» Ð¸ Ð²Ñ\213Ð²Ð¾Ð´Ð¸Ñ\202 Ð¸Ñ\205 Ð½Ð° Ñ
\215Ð°Ñ\200Ð°Ð½'. "
        "Ð\222Ð¾Ñ\202 Ñ\201Ð¿Ð¸Ñ\201Ð¾Ð° Ñ\203Ð¶Ðµ Ñ\201Ð´ÐµÐ»Ð°Ð½Ð½Ñ\213Ñ\205 Ð•Ð°Ð´Ð°Ñ
\207 (Ð½Ðµ Ð¿Ð¾Ð²Ñ\202Ð¾Ñ\200Ñ\217Ð¹ Ð¸Ñ\205):\n"
        f"{done_text}\n"
        "Ð\235Ð°Ð¿Ð¸Ñ\210Ð¸ Ð¢Ð\236Ð\233Ð¬Ð\232Ð\236 Ñ\202ÐµÐ°Ñ\201Ñ\202 Ð•Ð°Ð´Ð°Ñ\207Ð¸
Ð² Ð¾Ð´Ð½Ñ\203 Ñ\201Ñ\202Ñ\200Ð¾Ñ\207Ð°Ñ\203. Ð\221ÐµÐ• Ð¾Ð±Ñ\212Ñ\217Ñ\201Ð½ÐµÐ½Ð¸, Ð±
ÐµÐ• Ñ\201Ð¿Ð¸Ñ\201Ð°Ð¾Ð², Ð±ÐµÐ• 'Ð¿Ð¾Ñ\202Ñ\200ÐµÐ±Ñ\203ÐµÑ\202Ñ\201Ñ\217 5 Ð¼Ð¸Ð½Ñ\203
Ñ\202'."
    )

    print(f"[IDEALIST] Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð°Ð° Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð° Ð² Ollama ({MODEL})
...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True,
            text=True,
            timeout=300
        )
    except subprocess.TimeoutExpired:
        print("[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð½ÐµÐ¾ Ð¾Ñ\202Ð²ÐµÑ\202Ð¸Ð»Ð° Ð•Ð° 5 Ð
¼Ð¸Ð½Ñ\203Ñ\202.")
        return None
    except Exception as e:
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð²Ñ\213Ð•Ð¾Ð²Ð²ÐµÐ¾ Ollama: {e}")
        return None

    if result.returncode != 0:
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð²ÐµÑ\200Ð½Ñ\203Ð»Ð° Ð°Ð¾Ð´ {result
.returncode}.")
        return None

    idea = result.stdout.strip()
    if not idea:
        print("[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ollama Ð²ÐµÑ\200Ð½Ñ\203Ð»Ð° Ð¿Ñ\203Ñ\201Ñ
\202Ð¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202.")
        return None

    # Ð\222Ð«Ð\223Ð Ð£Ð\226Ð\220Ð\225Ð\234 Ð\234Ð\236Ð\224Ð\225Ð\233Ð¬, Ð§Ð¢Ð\236Ð\221Ð«
Ð\236Ð¡Ð\222Ð\236Ð\221Ð\236Ð\224Ð\230Ð¢Ð¬ Ð\237Ð\220Ð\234Ð¯Ð¢Ð¬
    subprocess.run(["ollama", "stop", MODEL], capture_output=True)
```

```
    print(f"[IDEALIST] Ð\234Ð¾Ð´ÐµÐ»Ñ\214 {MODEL} Ð²Ñ\213Ð³Ñ\200Ñ\203Ð¶ÐµÐ½Ð° Ð¸Ð• Ð¿Ð°Ð¼Ñ\217Ñ\202Ð¸.")

    return idea

def main():
    print("[IDEALIST] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° Ð\236Ñ\202Ð´ÐµÐ»Ð° Ð\230Ð´ÐµÐ°Ð»Ð¸Ñ\201Ñ\202Ð¾Ð²...")

    if not os.path.exists(FACTORY_DIR):
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¿Ð°Ð¿Ð°Ð° Ñ\204Ð°Ð±Ñ\200Ð¸Ð°Ð¸ Ð½Ðµ Ð½Ð°Ð¹Ð´ÐµÐ½Ð°: {FACTORY_DIR}")
        return

    done_tasks = get_success_tasks()
    print(f"[IDEALIST] Ð£Ð¶Ðµ Ñ\201Ð´ÐµÐ»Ð°Ð½Ð¾ Ð•Ð°Ð´Ð°Ñ\207: {len(done_tasks)}")

    idea = generate_idea(done_tasks)
    if not idea:
        print("[IDEALIST] Ð\235Ðµ Ñ\203Ð´Ð°Ð»Ð¾Ñ\201Ñ\214 Ñ\201Ð³ÐµÐ½ÐµÑ\200Ð¸Ñ\200Ð¾Ð²Ð°Ñ\202Ñ\214 Ð¸Ð´ÐµÑ\216. Ð\236Ñ\201Ñ\202Ð°Ð½Ð¾Ð²Ð°Ð°.")
        return

    print(f"\n[IDEALIST] Ð\237Ñ\200Ð¸Ð´Ñ\203Ð¼Ð°Ð½Ð° Ð•Ð°Ð´Ð°Ñ\207Ð°:\n{idea}\n")

    try:
        with open(TASK_FILE, "w", encoding="utf-8") as f:
            f.write(idea)
        print(f"[IDEALIST] Ð\227Ð°Ð´Ð°Ñ\207Ð° Ð•Ð°Ð¿Ð¸Ñ\201Ð°Ð½Ð° Ð² task.txt")
    except Exception as e:
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð•Ð°Ð¿Ð¸Ñ\201Ð¸ task.txt: {e}")
        return

    print("[IDEALIST] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð° director.py...")
    try:
        result = subprocess.run(
            [sys.executable, DIRECTOR_SCRIPT],
            cwd=FACTORY_DIR,
            timeout=3600
        )
        if result.returncode != 0:
            print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: director.py Ð•Ð°Ð²ÐµÑ\200Ñ\210Ð¸Ð»Ñ\201Ñ\217 Ñ\201 Ð°Ð¾Ð´Ð¾Ð¼ {result.returncode}.")
        else:
            print("[IDEALIST] director.py Ñ\203Ñ\201Ð¿ÐµÑ\210Ð½Ð¾ Ð•Ð°Ð²ÐµÑ\200Ñ\210Ð¸Ð»Ñ\200Ð°Ð±Ð¾Ñ\202Ñ\203.")
    except subprocess.TimeoutExpired:
        print("[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: director.py Ð½Ðµ Ð¾Ñ\202Ð²ÐµÑ\202Ð¸Ð» Ð•Ð°1 Ñ\207Ð°Ñ\201.")
    except Exception as e:
        print(f"[IDEALIST] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ð¿Ñ\200Ð¸ Ð•Ð°Ð¿Ñ\203Ñ\201Ð°Ðµ director.py: {e}")

if __name__ == "__main__":
    main()
-e


=== FILE: ./analyzer.py ===
import os
import subprocess
import re

FACTORY_DIR = os.path.expanduser("~/takcas_factory")
TASK_FILE = os.path.join(FACTORY_DIR, "task.txt")
DRAFT_1 = os.path.join(FACTORY_DIR, "draft_1.py")
DRAFT_ANALYZED = os.path.join(FACTORY_DIR, "draft_analyzed.py")
MODEL = "qwen2.5-coder:3b"

def clean_code(text):
```

```
    match = re.search(r'```(?:python)?\s*(.*?)```', text, re.DOTALL)
    if match:
        return match.group(1).strip()
    return text.strip()

def main():
    print("[ANALYZER] Ð\227Ð°Ð¿Ñ\203Ñ\201Ð°...")
    if not os.path.exists(TASK_FILE) or not os.path.exists(DRAFT_1):
        print("[ANALYZER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð½ÐµÑ\202 task.txt Ð¸Ð»Ð¸ draft_1.py.")
        return
    with open(TASK_FILE, "r", encoding="utf-8") as f:
        task = f.read().strip()
    with open(DRAFT_1, "r", encoding="utf-8") as f:
        code = f.read().strip()

    prompt = (
        "Ð¢Ñ\213 â\200\224 Ð°Ð½Ð°Ð»Ð¸Ñ\202Ð¸Ðº. Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ñ\214 Ð°Ð¾Ð´ Ð½Ð° Ñ
\201Ð¾Ð¾Ñ\202Ð²ÐµÑ\202Ñ\201Ñ\202Ð²Ð¸Ðµ Ð•Ð°Ð´Ð°Ñ\207Ðµ Ð¸ Ð¸Ñ\201Ð¿Ñ\200Ð°Ð²Ñ\214 Ñ\201Ð¸
Ð½Ñ\202Ð°Ð°Ñ\201Ð¸Ñ\207ÐµÑ\201Ð°Ð¸Ðµ Ð¾Ñ\210Ð¸Ð±Ð°Ð¸. "
        "Ð\222ÐµÑ\200Ð½Ð¸ Ð¢Ð\236Ð\233Ð¬Ð\232Ð\236 Ð¸Ñ\201Ð¿Ñ\200Ð°Ð²Ð»ÐµÐ½Ð½Ñ\213Ð¹ Ð°Ð¾
Ð´, Ð±ÐµÐ• Ð¾Ð±Ñ\212Ñ\217Ñ\201Ð½ÐµÐ½Ð¸Ð¹. "
        "Ð\222Ð\220Ð\226Ð\235Ð\236: Ð\230Ñ\201Ð¿Ð¾Ð»Ñ\214Ð•Ñ\203Ð¹ Ð¢Ð\236Ð\233Ð¬Ð\232Ð
\236 Ñ\201Ñ\202Ð°Ð½Ð´Ð°Ñ\200Ñ\202Ð½Ñ\213Ðµ Ð±Ð¸Ð±Ð»Ð¸Ð¾Ñ\202ÐµÐ°Ð¸ Python. "
        f"Ð\227Ð°Ð´Ð°Ñ\207Ð°: {task}\n"
        f"Ð\232Ð¾Ð´:\n{code}"
    )

    print(f"[ANALYZER] Ð\236Ñ\202Ð¿Ñ\200Ð°Ð²Ð°Ð° Ð•Ð°Ð¿Ñ\200Ð¾Ñ\201Ð° Ð² Ollama ({MODEL})
...")
    try:
        result = subprocess.run(
            ["ollama", "run", MODEL, prompt],
            capture_output=True, text=True, timeout=600
        )
    except subprocess.TimeoutExpired:
        print("[ANALYZER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ñ\202Ð°Ð¹Ð¼Ð°Ñ\203Ñ\202.")
        return
    except Exception as e:
        print(f"[ANALYZER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: {e}")
        return

    if result.returncode != 0:
        print(f"[ANALYZER] Ð\236Ñ\210Ð¸Ð±Ð°Ð° Ollama: Ð°Ð¾Ð´ {result.returncode}")
        return

    clean = clean_code(result.stdout)
    if not clean:
        print("[ANALYZER] Ð\236Ñ\210Ð¸Ð±Ð°Ð°: Ð¿Ñ\203Ñ\201Ñ\202Ð¾Ð¹ Ð¾Ñ\202Ð²ÐµÑ\202.")
        return

    with open(DRAFT_ANALYZED, "w", encoding="utf-8") as f:
        f.write(clean)
    print(f"[ANALYZER] Ð\232Ð¾Ð´ Ñ\201Ð¾Ñ\205Ñ\200Ð°Ð½ÐµÐ½ Ð² draft_analyzed.py")

if __name__ == "__main__":
    main()
-e


=== FILE: ./draft_1.py ===
-e


=== FILE: ./draft_analyzed.py ===
import threading
import time
from collections import OrderedDict

class TTLCache:
    def __init__(self, capacity: int, default_ttl: float, cleanup_interval:\033[17D\033[K
```

```
cleanup_interval: float):
        self.capacity = capacity
        self.default_ttl = default_ttl
        self.cleanup_interval = cleanup_interval
        self.cache = OrderedDict()
        self.lock = threading.Lock()
        self.hits = 0
        self.misses = 0
        self.evictions = 0
        self.expirations = 0
        self.on_evict = None

    def get(self, key):
        with self.lock:
            if key in self.cache:
                self.hits += 1
                self._update_ttl(key)
                return self.cache[key]
            else:
                self.misses += 1
                return None

    def put(self, key, value, ttl=None):
        with self.lock:
            if key in self.cache:
                self.evictions += 1
                self.cache.pop(key)
            self._update_ttl(key, ttl or self.default_ttl)
            self.cache[key] = value
            if len(self.cache) > self.capacity:
                self._evict_lru()

    def _update_ttl(self, key, ttl=None):
        if ttl is not None:
            self.cache.move_to_end(key, last=False)
        else:
            self.cache.move_to_end(key)

    def _evict_lru(self):
        with self.lock:
            while len(self.cache) > self.capacity:
                key, _ = self.cache.popitem(last=False)
                self.expirations += 1
                if self.on_evict:
                    self.on_evict(key, self.cache[key], "lru")

    def start_cleanup(self):
        while True:
            with self.lock:
                if time.time() - self.last_cleanup > self.cleanup_interval:\033[22D\033[K
self.cleanup_interval:
                    self._evict_expired()
            time.sleep(self.cleanup_interval)

    def _evict_expired(self):
        with self.lock:
            for key in list(self.cache.keys()):
                if time.time() - self.cache[key] > self.default_ttl:
                    self.evictions += 1
                    if self.on_evict:
                        self.on_evict(key, self.cache[key], "ttl")
                    self.cache.pop(key)

    def get_stats(self):
        with self.lock:
            return {
                "hits": self.hits,
                "misses": self.misses,
                "evictions": self.evictions,
                "expirations": self.expirations,
```

```
                "size": len(self.cache),
                "hit_ratio": self.hits / (self.hits + self.misses) if self.\033[5D\033[K
self.hits + self.misses > 0 else 0
            }

    def set_on_evict(self, callback):
        self.on_evict = callback

# Ð¢ÐµÑ\201Ñ\202
if __name__ == "__main__":
    cache = TTLCache(capacity=100, default_ttl=0.5, cleanup_interval=0.1)
    cache.set_on_evict(lambda key, value, reason: print(f"Evicted: {key} du\033[2D\033[K
due to {reason}"))

    threads = []
    for _ in range(8):
        thread = threading.Thread(target=lambda: [
            cache.get(i % 50),
            cache.put(i % 50, i, 0.05 + 0.15 * (i % 50))
            for i in range(500)
        ])
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    stats = cache.get_stats()
    assert stats["hits"] + stats["misses"] == 4000
    assert stats["size"] <= 100
    assert not cache.on_evict_called  # Ð\237Ñ\200Ð¾Ð²ÐµÑ\200Ð°Ð° Ñ\202Ð¾Ð³Ð¾, Ñ\207Ñ\202
Ð¾ on_evict Ð½Ðµ Ð²Ñ\213Ð•Ñ\213\033[4D\033[K
Ð²Ñ\213Ð•Ñ\213Ð²Ð°Ð»Ñ\201Ñ\217 Ð¿Ð¾Ð´ Ð»Ð¾Ð°Ð¼Ð¼

    print(stats)-e


=== FILE: ./logic_errors.txt ===
-e


=== FILE: ./syntax_errors.txt ===
-e


=== FILE: ./coder_errors.txt ===
-e


=== FILE: ./draft_final.py ===
import threading
import time
from collections import OrderedDict

class TTLCache:
    def __init__(self, capacity: int, default_ttl: float, cleanup_interval:\033[17D\033[K
cleanup_interval: float):
        self.capacity = capacity
        self.default_ttl = default_ttl
        self.cleanup_interval = cleanup_interval
        self.cache = OrderedDict()
        self.lock = threading.Lock()
        self.hits = 0
        self.misses = 0
        self.evictions = 0
        self.expirations = 0
        self.on_evict = None

    def get(self, key):
        with self.lock:
```

```
            if key in self.cache:
                self.hits += 1
                self._update_ttl(key)
                return self.cache[key]
            else:
                self.misses += 1
                return None

    def put(self, key, value, ttl=None):
        with self.lock:
            if key in self.cache:
                self.evictions += 1
                self.cache.pop(key)
            self._update_ttl(key, ttl or self.default_ttl)
            self.cache[key] = value
            if len(self.cache) > self.capacity:
                self._evict_lru()

    def _update_ttl(self, key, ttl=None):
        if ttl is not None:
            self.cache.move_to_end(key, last=False)
        else:
            self.cache.move_to_end(key)

    def _evict_lru(self):
        with self.lock:
            while len(self.cache) > self.capacity:
                key, _ = self.cache.popitem(last=False)
                self.expirations += 1
                if self.on_evict:
                    self.on_evict(key, self.cache[key], "lru")

    def start_cleanup(self):
        while True:
            with self.lock:
                if time.time() - self.last_cleanup > self.cleanup_interval:\033[22D\033[K
self.cleanup_interval:
                    self._evict_expired()
            time.sleep(self.cleanup_interval)

    def _evict_expired(self):
        with self.lock:
            for key in list(self.cache.keys()):
                if time.time() - self.cache[key] > self.default_ttl:
                    self.evictions += 1
                    if self.on_evict:
                        self.on_evict(key, self.cache[key], "ttl")
                    self.cache.pop(key)

    def get_stats(self):
        with self.lock:
            return {
                "hits": self.hits,
                "misses": self.misses,
                "evictions": self.evictions,
                "expirations": self.expirations,
                "size": len(self.cache),
                "hit_ratio": self.hits / (self.hits + self.misses) if self.\033[5D\033[K
self.hits + self.misses > 0 else 0
            }

    def set_on_evict(self, callback):
        self.on_evict = callback

# Ð¢ÐµÑ\201Ñ\202
if __name__ == "__main__":
    cache = TTLCache(capacity=100, default_ttl=0.5, cleanup_interval=0.1)
    cache.set_on_evict(lambda key, value, reason: print(f"Evicted: {key} du\033[2D\033[K
due to {reason}"))
```

```
    threads = []
    for _ in range(8):
        thread = threading.Thread(target=lambda: [
            cache.get(i % 50),
            cache.put(i % 50, i, 0.05 + 0.15 * (i % 50))
            for i in range(500)
        ])
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    stats = cache.get_stats()
    assert stats["hits"] + stats["misses"] == 4000
    assert stats["size"] <= 100
    assert not cache.on_evict_called  # Đ\237Ñ\200Đ¾Đ²ĐµÑ\200Đ°Đ° Ñ\202Đ¾Đ³Đ¾, Ñ\207Ñ\202
Đ¾ on_evict Đ½Đµ Đ²Ñ\213Đ•Ñ\213\033[4D\033[K
Đ²Ñ\213Đ•Ñ\213Đ²Đ°Đ»Ñ\201Ñ\217 Đ¿Đ¾Đ´ Đ»Đ¾Đ°Đ¾Đ¼

    print(stats)-e
```